

Google Summer of Code 2026 Proposal

Neural Operators for Fast Simulation of Strong Gravitational Lensing

DeepLense · ML4SCI

Michael Toomey · Sergei Gleyzer · Pranath Reddy · Ashutosh Ojha · J Rishi

Paras Balani

Birla Institute of Technology and Science, Pilani, Hyderabad Campus

M.Sc. Mathematics & B.E. Computer Science (Dual Degree)

f20230738@hyderabad.bits-pilani.ac.in · github.com/Metrex-Nova · [LinkedIn](#)

Contents

1	Project Explanation	3
2	Technical Understanding required for the project	5
2.1	The Lensing Forward Problem	5
2.2	Why not CNNs	5
2.3	The Savior FNO	6
3	Relevant Experience	7
3.1	Neural Operator Surrogate for Rotating NFW Wormhole Lensing.	7
3.1.1	Dataset generation	7
3.1.2	1D FNO	8
3.1.3	MLP and CNN	8
3.1.4	The FNO training pipeline	9
3.1.5	The Baseline Training Setup	10
3.2	Physics-Informed Deep Learning for Lensing Inverse Problems.	11
3.2.1	Poisson Solver	11
3.2.2	Lens Equation	11
3.2.3	Deflection field	12
3.2.4	Convolutional Neural Network	12
3.2.5	UNet Architecture	12
3.3	Spectral Latent Dynamics for PDE Systems.	14
3.3.1	Train Auto Encoder	14
3.3.2	Train dynamics	15
3.3.3	Train FNO	15
3.4	Traversable Wormholes with Van der Waals Equation of State in $f(R, L, T)$ Gravity	16
3.5	Ray Tracing and Shadows of Dark-Matter-Supported Wormholes and Gravastars (In-progress)	17
4	Tests	19
4.1	Common Test I: Multi-Class Classification.	19
4.1.1	Model Architecture	19
4.1.2	Data Pipeline and Augmentation	20
4.1.3	Two-Phase Training	20
4.1.4	Results	21
4.2	Specific Test IV: Neural Operator Classifier - My journey from 0.77 to 0.9439 AUC	23
4.2.1	Architecture 11a -FNOEfficientNet	23
4.2.2	Architecture 1 - Pure FNO2d (Standard Implementation)	24
4.2.3	Architecture 2 - FNO2d with Gaussian Weight Initialization	24
4.2.4	Architecture 3 - FNO2d with Coordinate Grid Embedding	25

4.2.5	Architecture 4 - WideResFNO (BatchNorm + Increased Capacity)	25
4.2.6	Architecture 5 - DeepLenseSpecialist (The King)	26
4.2.7	Architecture 6 - DeepLenseV3 (Third Spectral Block + Residual Shortcut)	26
4.2.8	Architecture 7 - DualPathModel and HybridSpecialist (EfficientNet + FNO)	27
4.2.9	Architecture 8 - SideCar95 (Frozen King + B0 Residual Refiner)	28
4.2.10	Architecture 9 - DeepLenseBypassV2 (Manual Weight Transplant)	28
4.2.11	Architecture 10 - FNOEfficientNet	29
4.2.12	Discussion	31
4.2.13	Summary and Comparative Results	32
4.2.14	Comparison with Common Test I Baseline	33
5	Proposed Approach	34
5.1	Dataset generation.	34
5.2	FNO implementation.	34
5.3	Baseline comparison.	34
5.4	Evaluation.	35
6	Implementation Plan & Timeline	37
7	Why I Am a Good Fit	40

General Information

Organization DeepLense, Machine Learning for Science (ML4SCI)
Project Neural Operators for Fast Simulation of Strong Gravitational Lensing

Mentors

- Michael Toomey (Massachusetts Institute of Technology)
- Sergei Gleyzer (University of Alabama)
- Pranath Reddy (Independent Researcher)
- Ashutosh Ojha (IIT (BHU) Dhanbad)
- J Rishi (Indian Institute of Science, Bangalore)

Student Contributor

Name	Paras Balani
University	BITS Pilani, Hyderabad Campus (2023–2028)
Degree	M.Sc. Mathematics & B.E. Computer Science (Dual Degree), Minor in Data Science
CGPA	8.74 / 10.0
Email	f20230738@hyderabad.bits-pilani.ac.in
Phone	+91-8529818697
GitHub	github.com/Metrex-Nova
LinkedIn	linkedin.com/in/pbalani-mnova
Location	Hyderabad, India
Time Zone	IST (UTC+5:30)
Availability	May 15 – July 31, 2026 (350-hour track)

Project Explanation

- Strong gravitational lensing is one of the most direct observational inputs for dark matter structure and the large-scale mass distribution of the universe.
- Simulating lensed images, given a lens mass distribution and a background source requires solving the lens equation via ray tracing which is accurate but computationally expensive.
- Tools such as LENSTRONOMY evaluate the deflection field point by point across the image plane, making large-scale parameter sweeps, Bayesian inference, and real-time analysis prohibitively slow.
- This proposal aims to build a **neural operator surrogate** that learns the functional mapping

$$\mathcal{G} : \kappa(\boldsymbol{\theta}) \longmapsto I_{\text{lensed}}(\boldsymbol{\theta}),$$

where κ is the convergence field which is a dimensionless scalar field which encodes the projected surface mass density of the gravitational lens.

- Physically, κ represents how matter is distributed across the lens plane after integrating along the line of sight, normalized by a critical density scale.
- Regions where κ is higher correspond to stronger gravitational influence, leading to more pronounced bending of light rays. As a result, κ directly determines how incoming light from a distant source is distorted.
- The observable outcome of this distortion is the lensed image, denoted by I_{lensed} , which may exhibit characteristic features such as arcs, rings, or multiple images depending on the underlying mass distribution.

- In traditional approaches, convolutional neural networks (CNNs) are commonly used to model mappings between inputs and outputs. However, these networks inherently operate on fixed-resolution grids and learn relationships between finite-dimensional vectors. This creates a limitation when dealing with physical systems like gravitational lensing, where the underlying processes are continuous and governed by differential equations.
- Specifically, the lensing phenomenon is described by equations derived from gravitational potential theory, which can be expressed in the form of partial differential equations (PDEs). These equations define a mapping not just between discrete data points, but between entire functions.
- Neural operators, and in particular Fourier Neural Operators (FNOs), address this limitation by learning mappings between function spaces rather than fixed-size vectors[3]. Instead of approximating a function at a single resolution, FNOs learn an operator that can act on functions represented at arbitrary resolutions.
- This is achieved by transforming the input into the Fourier domain, where global structures and long-range dependencies are more naturally captured, applying learned transformations to a truncated set of Fourier modes, and then mapping back to physical space.
- Because of this formulation, FNOs are inherently resolution-invariant and can generalize across different discretizations of the same underlying physical system.
- The lens equation defines a continuous mapping from the convergence field κ to the observed image I_{lensed} , effectively acting as a solution operator of a PDE system.
- By learning this operator directly, FNOs bypass the need for repeated numerical solutions of the governing equations, enabling faster inference while maintaining consistency with the physics. Consequently, they provide a principled and efficient framework for modeling lensing phenomena in a way that aligns closely with the mathematical structure of the problems.
- The primary deliverable of this project is a trained Fourier Neural Operator (FNO) based surrogate model for rapid generation of gravitationally lensed images from input convergence fields. Once trained, the model enables efficient inference, making it suitable for large-scale parameter exploration and time-sensitive analysis tasks.
- The model can then be tested for a range of astrophysical scenarios like variations in lens mass profiles, source and lens redshifts, and intrinsic source gradients, etc..
- The urgency of this problem is driven by the Vera Rubin Observatory’s Legacy Survey of Space and Time (LSST), which is projected to discover $\mathcal{O}(10^5)$ strong lensing systems over its ten-year baseline [1]. Traditional ray-tracing solvers that require seconds per evaluation are incompatible with the inference demands of this sample: Bayesian lens modeling at even 10^4 evaluations per system would require $\mathcal{O}(10^9)$ solver calls across the full LSST catalog. Neural operator surrogates that achieve $> 10^3\times$ speedup are not an optimization; they are a prerequisite for science at LSST scale.

The Lensing Forward Problem

- Note that, under the thin-lens approximation, the three-dimensional gravitational lensing problem reduces to an effective two-dimensional mapping on the lens plane.
- So, we assume all deflections to occur instantaneously at a single plane perpendicular to the line of sight.
- A light ray emitted from a source at angular position β is observed at an image-plane position θ according to the lens equation:

$$\beta = \theta - \alpha(\theta),$$

where $\alpha(\theta)$ is the deflection angle induced by the gravitational potential of the lens. So, basically the deflection would encode the mass distribution along the line of sight.

- Now, to get the deflection angle we use 2 relations, (i) the relation between lensing potential and convergence field and (ii) the relation between lensing potential and deflection angle. So, lensing is basically a gradient-driven distortion of light paths.

$$\nabla^2 \psi(\theta) = 2\kappa(\theta).$$

$$\alpha(\theta) = \nabla \psi(\theta),$$

- This means that we just need to first solve the Poisson equation to recover the lensing potential $\psi(\theta)$ from the given convergence field $\kappa(\theta)$ by using FFTs (fast fourier transforms) and then calculating the deflection field $\alpha(\theta)$ by taking spatial derivatives of the potential. And then we will trace back the rays backward from the image plane to the source plane using the lens equation, and the source intensity is interpolated at the mapped positions to construct the observed lensed image.
- From a computational perspective, this pipeline is expensive. A naive implementation requires evaluating quantities across an $N \times N$ grid, leading to an $O(N^2)$ cost for ray-shooting and interpolation steps. The dominant cost arises from solving the Poisson equation for the lensing potential: direct methods scale as $O(N^4)$, while in practice FFT-based solvers reduce this to $O(N^2 \log N)$.
- So, parameter inference typically demands 10^4 – 10^6 such evaluations, making the overall procedure computationally prohibitive. In practical applications such as Bayesian inference or parameter estimation, where the forward model must be evaluated repeatedly (often 10^4 – 10^6 times), this computational burden becomes a significant bottleneck. As a result, efficient approximations to this forward operator are essential for enabling scalable analysis of strong lensing systems.

Why not CNNs

- The first and the foremost issue is that a standard convolutional neural network (CNN) trained on paired samples (κ_i, I_i) can learn an effective mapping from the convergence field to the corresponding lensed image at a FIXED resolution.
- And the mapping is more or less tied to the resolution it is trained on. That is a CNN trained on 64×64 inputs cannot be directly applied to 128×128 inputs without architectural modification or retraining. The possible reasons for this can be that the learned filters and feature hierarchies are resolution-dependent.
- And the issue arises when the underlying problems for the parametric studies are defined on a continuous space and that different resolutions correspond merely to different discretizations of the same physical system.
- More fundamentally, CNNs do not explicitly account for the operator nature of the forward lensing problem. They treat each input-output pair as an independent mapping between finite-dimensional vectors, without recognizing that these pairs arise from a shared underlying functional relationship.

- In the context of gravitational lensing, the mapping from $\kappa(\boldsymbol{\theta})$ to $I_{\text{lensed}}(\boldsymbol{\theta})$ is not arbitrary, but is governed by a system of differential equations. Ignoring this structure can limit generalization, particularly when the model is evaluated on configurations that differ from the training distribution.

The Savior FNO

- Fourier Neural Operators (FNOs) solve these problems in a very interesting way. They learn the mappings between function spaces rather than fixed-dimensional vectors[2]. That is each FNO layer applies a learned linear transformation in the spectral domain: the input is transformed via a Fourier transform, a subset of Fourier modes is multiplied by a trainable weight tensor, and the result is mapped back to physical space.
- And because the transformation is defined in terms of Fourier modes rather than discrete pixel locations, the learned operator is inherently resolution-invariant and can be evaluated on different grid sizes without retraining.
- This formulation is particularly well-aligned with the structure of the lensing forward problem. The governing Poisson equation is naturally expressed in terms of differential operators, which in Fourier space correspond to simple algebraic relationships between modes.
- By operating directly in the spectral domain, FNOs implicitly mirror this structure, enabling them to approximate the underlying solution operator in a manner consistent with the physics of the problem.
- This formulation is consistent with the theoretical framework of neural operators, which have been shown to be universal approximators of nonlinear continuous operators between function spaces [3].

Relevant Experience

This project builds upon the foundational neural operator frameworks established in the literature[4] [3] [2]. My background sits at the intersection of gravitational physics and deep learning, and my prior projects map directly onto the components of this proposal.

Neural Operator Surrogate for Rotating NFW Wormhole Lensing.

[GitHub](#)

- Developed a Fourier Neural Operator (FNO) surrogate model to accelerate gravitational lensing simulations of slowly rotating traversable wormholes embedded in an NFW dark matter halo.
- Generated a dataset of 2000 intensity profiles by integrating null geodesics through a MorrisThorne-type metric modified by the NFW potential and frame-dragging effects, using adaptive RungeKutta methods with event detection.
- Implemented the FNO architecture with 4 spectral convolution layers, achieving a test MSE of 1.60×10^{-3} and an 18.5% relative L^2 error, outperforming an MLP baseline with $4\times$ more parameters.
- Achieved a $6,392\times$ inference speedup over the physics solver, enabling real-time parameter exploration across throat radii and spin parameters.
- Identified critical architectural constraints for neural operators in sparse intensity regimes, including the detrimental effect of instance normalization and spectral mode limitations, providing insights for astrophysical applications of neural operators.

3.1.1 Dataset generation

```
RO_MIN, RO_MAX = 0.3, 2.5
J_MIN, J_MAX = 0.0, 1.5
rng = np.random.default_rng(seed)
r0_arr = rng.uniform(RO_MIN, RO_MAX, n_samples)
J_arr = rng.uniform(J_MIN, J_MAX, n_samples)
params = np.stack([r0_arr, J_arr], axis=1).astype(np.float32)
```

So first I defined the physical bounds and generated the input parameter space.

```
def _worker(args):
    i, r0, J, b_grid = args
    try:
        I, b_ph = compute_intensity_profile(r0, J, b_grid)
    except Exception:
        I, b_ph = np.zeros(len(b_grid), dtype=np.float32), 0.0
    return i, I.astype(np.float32), float(b_ph), time.perf_counter() - t0
```

Then I defined the worker for multiprocessing for faster generation of data. Note that, I wrapped the calculation in try-catch block so as to prevent the one failed ray-trace from crashing the entire pool of rays handled by the worker.

```
with mp.Pool(processes=n_cpu) as pool:
    with tqdm(total=n_samples, desc="Generating", ...) as pbar:
        for result in pool.imap_unordered(_worker, jobs, chunksize=4):
            i, I, b_ph, elapsed = result
            profiles[i] = I
            b_ph_arr[i] = b_ph
            pbar.update(1)
```

So, then I distributed the tasks among the available cores of the system. Here, I used the imap unordered function with a chunksize of 4 to optimizes the trade-off between inter-process communication overhead and worker utilization.

3.1.2 1D FNO

```
def forward(self, x):
    N = x.shape[-1]
    x_ft = torch.fft.rfft(x)
    w = torch.complex(self.w_re, self.w_im)
    out_ft = torch.zeros(..., dtype=torch.cfloat)
    out_ft[:, :, :self.modes] = torch.einsum("bim,iom->bom",
                                              x_ft[:, :, :self.modes], w)
    return torch.fft.irfft(out_ft, n=N)
```

So, first we transform the given input to frequency domain using Real Fast Fourier Transform (RFFT). Then we define the learnable weights in the complex plane and filter out the low-frequency modes and then move back to the spatial domain. Basically, this act of "truncating" the higher frequencies acts as a learned low-pass filter, allowing the model to focus on the global structure of the intensity profile rather than local noise.

```
class FNOBlock(nn.Module):
    def __init__(self, width, modes):
        super().__init__()
        self.spectral = SpectralConv1d(width, width, modes)
        self.w = nn.Conv1d(width, width, kernel_size=1)

    def forward(self, x):
        return F.gelu(self.spectral(x) + self.w(x))
```

So clearly as written in the code, we can see that each FNO layer is a combination of two paths. The spectral path, which captures the global dependencies across the entire grid and the nn.conv path which acts as a linear transform to maintain local spatial information. Together they ensure that the network can learn both broad physical trends and sharp local peaks.

```
class FNO1d(nn.Module):
    def forward(self, x):
        x = self.fc0(x)          # Project input to higher-dimensional 'width'
        x = x.permute(0, 2, 1)   # Change shape to [Batch, Channels, Spatial]
        for block in self.blocks:
            x = block(x)         # Deep spectral processing
        x = x.permute(0, 2, 1)   # Back to [Batch, Spatial, Channels]
        x = F.gelu(self.fc1(x))  # Final regression head
        return self.fc2(x).squeeze(-1)
```

The FNO1d class manages the lifecycle of the data. It first lifts the input features (like coordinate and parameters) into a high-dimensional latent space (fc0). After several iterations through spectral blocks, it projects the latent features back down to the target output (fc2). The use of permute is critical here, as it switches between the "feature-first" format required by Linear layers and the "channel-first" format required by Spectral/Conv1d layers.

3.1.3 MLP and CNN

```
class MLPBaseline(nn.Module):
    def __init__(self, n_params=2, n_out=150, hidden=512, n_layers=4):
        super().__init__()
        layers = [nn.Linear(n_params, hidden), nn.GELU()]
        for _ in range(n_layers - 1):
            layers += [nn.LayerNorm(hidden), nn.Linear(hidden, hidden), nn.GELU()]
        layers.append(nn.Linear(hidden, n_out))
        self.net = nn.Sequential(*layers)
```

The MLPBaseline uses a "fully connected" architecture. It treats the output intensity profile as a single flat vector. By using LayerNorm and GELU activation functions, the model is kept numerically stable during

training. While powerful for small-scale problems, this model is "spatially unaware". So, basically it doesn't know that point 5 on the grid is physically next to point 6.

```
self.decoder = nn.Sequential(
    nn.ConvTranspose1d(self.init_ch, 128, kernel_size=4, stride=4), nn.GELU(),
    nn.ConvTranspose1d(128, 64, kernel_size=4, stride=4), nn.GELU(),
    nn.Conv1d(32, 1, kernel_size=3, padding=1),
)
```

Now, unlike MLPBaseline, CNNBaseline uses a decoder structure which is common in image generation. It starts with a low-resolution "stem" and uses ConvTranspose1d layers to "upsample" the features. This allows the model to learn local spatial correlations and "smoothness" in the intensity profiles, which is a key characteristic of physical data.

```
def forward(self, params):
    x = self.stem(params)
    x = x.view(x.shape[0], self.init_ch, self.init_len)
    x = self.decoder(x)
    x = F.interpolate(x, size=self.n_out, mode='linear', align_corners=False)
    return x.squeeze(1)
```

Because the combinations of kernel sizes and strides in convolutions might not result in exactly 150 points, so I used F.interpolate in the final step. This ensures the model output perfectly aligns with the ground-truth data from the ray-tracing simulations, regardless of the internal CNN architecture.

3.1.4 The FNO training pipeline

```
optimizer = Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
scheduler = CosineAnnealingLR(optimizer, T_max=epochs, eta_min=lr * 0.01)
criterion = nn.MSELoss()
```

To optimize the FNO, I use the Adam optimizer paired with a Cosine Annealing Learning Rate Scheduler. This combination is effective for scientific machine learning; the Adam optimizer handles the complex gradient landscape of spectral convolutions, while the scheduler gradually reduces the learning rate to "fine-tune" the model as it approaches a minimum, preventing overshooting in the final epochs.

```
for fno_input, _, profile in train_loader:
    optimizer.zero_grad()
    loss = criterion(model(fno_input), profile)
    loss.backward()

    nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    optimizer.step()
```

During the training phase, I implemented Gradient Clipping (clip grad norm). In spectral methods, large gradients can sometimes occur due to the complex-valued multiplications in the Fourier domain. Clipping the gradients at a maximum norm of 1.0 ensures training stability and prevents the "exploding gradient" problem, which is critical when training deep FNO architectures.

```
if vl_loss < best_val:
    best_val = vl_loss
    torch.save({
        "epoch": epoch,
        "model_state": model.state_dict(),
        "config": dict(modes=modes, width=width, n_layers=n_layers),
        "val_mse": vl_loss,
    }, ckpt_path)
```

The pipeline includes an Automated Checkpointing system. It tracks the validation Mean Squared Error (MSE) at every epoch. If the current model outperforms all previous iterations, the full state, including

hyper-parameters and optimizer weights is saved to disk. This ensures that the "Best Model" is preserved even if the training overfits in later epochs.

3.1.5 The Baseline Training Setup

```
model = MLPBaseline(n_params=2, n_out=n_grid, hidden=hidden, n_layers=n_layers).to(device)
optimizer = Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
scheduler = CosineAnnealingLR(optimizer, T_max=epochs, eta_min=lr * 0.01)
criterion = nn.MSELoss()
```

By keeping the optimization hyperparameters constant, I can isolate the performance gains to the architectural differences (spectral convolutions vs. linear layers) rather than differences in the training regime.

```
for _, params, profile in train_loader:
    params, profile = params.to(device), profile.to(device)
    optimizer.zero_grad()
    loss = criterion(model(params), profile)
    loss.backward()
    optimizer.step()
```

The training loop demonstrates the direct mapping approach of the baseline. While the FNO utilizes the full input tensor (including the spatial grid), the MLPBaseline focuses on the mapping from the 2D physical parameter space ($\mathbb{R}^2 \rightarrow \mathbb{R}^{150}$). This snippet highlights the data flow from the make loaders utility to the model, ensuring memory-efficient batch processing on the GPU.

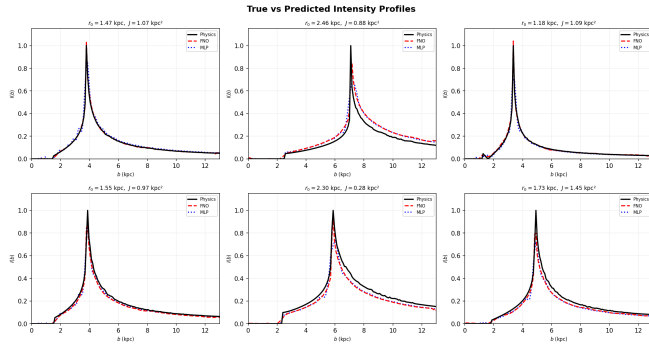


Figure 1: True vs. predicted intensity profiles for six representative test samples with varying throat radius r_0 and spin parameter J . The black solid line shows the physics solver result, the red dashed line the FNO prediction, and the blue dotted line the MLP baseline. The FNO more accurately captures the peak amplitude and tail decay, particularly for configurations with larger throat radii where the photon ring is broader and displaced to larger impact parameters.

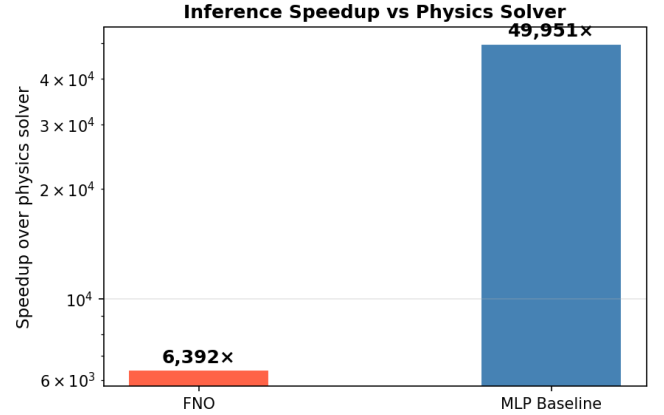


Figure 2: Inference speedup relative to the RK45 physics solver. The FNO achieves a 6,392 $\times$ speedup, while the MLP baseline achieves 49,951 $\times$. The vertical axis is logarithmic. Both surrogates enable real-time parameter exploration infeasible with the physics solver alone, with the FNO offering superior accuracy at a modest inference cost.

- Designed a fully differentiable forward lensing pipeline, spectral FFT-based Poisson solver ($\nabla^2\psi = 2\kappa$), deflection field computation, and ray-shooting, embedded as a physics-informed loss term in a U-Net trained to reconstruct convergence fields κ from lensed images (SIS+NFW synthetic dataset, 1000 samples, 128×128 grids).
- Physics-informed model achieved $2.5\times$ better re-lensing MSE (4.8×10^{-4} vs. 1.2×10^{-3}) than the baseline U-Net, demonstrating that the predicted mass maps are more physically consistent with the observed lensing signal despite higher pixel-wise convergence MSE.
- Near-flat noise degradation ($1.07\times$ MSE increase at $\sigma = 0.05$ vs. baseline's $3.6\times$), showing that the physics loss acts as implicit regularization by constraining outputs to the manifold of lens-equation-consistent convergence fields.
- Identified and analyzed failure modes including checkerboard artifacts from spectral-method boundary conditions and background offset from imbalanced pixel weighting, with proposed mitigations (low-pass filtering, multi-scale loss).

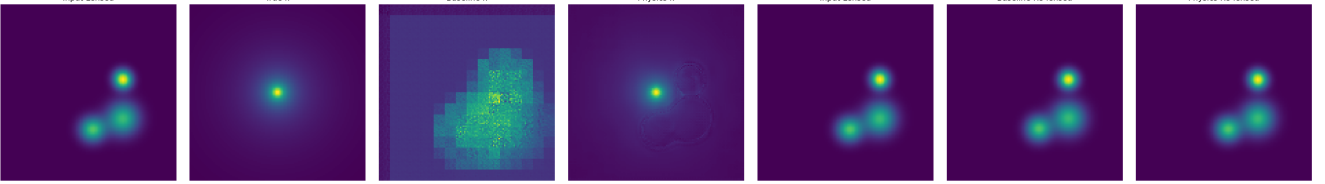


Figure 3: Side-by-side comparison of baseline (“Baseline κ ”) and physics-informed (“Physics κ ”) reconstructions. Left four panels: input lensed image, true κ , baseline predicted κ , and physics-informed predicted κ . Right three panels: input lensed image, re-lensed image through baseline κ , and re-lensed image through physics-informed κ . The physics-informed model produces a more structured (though noisier) κ map, and its re-lensed image more closely resembles the input.

3.2.1 Poisson Solver

```
kx, ky = _frequency_grid((H, W), pixel_scale=pixel_scale, device=device)
k2 = kx**2 + ky**2
k2[0, 0] = 1.0
kappa_hat = torch.fft.fft2(kappa)
psi_hat = -kappa_hat / k2
psi_hat[..., 0, 0] = 0.0
psi = torch.fft.ifft2(psi_hat).real
```

So, first I generate the spatial frequency grids (kx,ky) for the specific image size then calculate the square magnitude of the wave vector then handle the 'DC' component to prevent division by zero. Then I simply transform the convergence(kappa) to Fourier space and solve the Poisson equation and enforce the zero mena potential by nulling the DC component and then transform back to the real space to get the final lensing potential.

3.2.2 Lens Equation

```
grid = _normalized_meshgrid((H, W), device=device)
alpha_norm = torch.zeros_like(alpha)
alpha_norm[:, 0] = alpha[:, 0] * (2.0 / (W - 1)) # X-deflection
alpha_norm[:, 1] = alpha[:, 1] * (2.0 / (H - 1)) # Y-deflection
sampling_grid = grid - alpha_norm.permute(0, 2, 3, 1)
lensed = F.grid_sample(source, sampling_grid, mode="bilinear", padding_mode="border")
```

So, first I created a base co-ordinate grid for the image normalized from -1 to 1 then I simply converted the deflection field from pixels to normalized units because the deflection is in pixels so we must normalize

it to this range so the shift matches the grid's scale. Then I applied the lens equation which maps each 'lensed' pixel back to its original location in the source and then resampled the source image onto the new distorted grid using bilinear interpolation.

3.2.3 Deflection field

```
psi_hat = torch.fft.fft2(psi)
alpha_x_hat = 1j * kx * psi_hat
alpha_y_hat = 1j * ky * psi_hat
alpha = torch.stack([torch.fft.ifft2(alpha_x_hat).real,
                    torch.fft.ifft2(alpha_y_hat).real], dim=1)
```

Again, the same thing, I first transformed the potential to Fourier space then computed the derivatives by multiplying by frequencies ($i * k$) and then transformed them back to real space and stacked them as a vector field (x, y).

3.2.4 Convolutional Neural Network

```
self.net = nn.Sequential(
    nn.Conv2d(in_channels, num_filters, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),

    nn.Conv2d(num_filters, num_filters, kernel_size=3, padding=1),
    nn.BatchNorm2d(num_filters),
    nn.ReLU(inplace=True),

    nn.Conv2d(num_filters, num_filters, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),

    nn.Conv2d(num_filters, out_channels, kernel_size=1),
)
```

So, I designed a simple four-layer convolutional stack for feature extraction. The first layer is input layer which detects basic edges/shapes in the lensed arcs. The second layer is refinement layer which uses BatchNorm to stabilize learning of complex patterns and the third is refinement layer which refines the spatial features and finally the fourth layer projects it to the output map.

3.2.5 UNet Architecture

```
# The DoubleConv Block
self.net = nn.Sequential(
    nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
    nn.BatchNorm2d(out_channels), # Normalizes the layer to speed up convergence
    nn.ReLU(inplace=True),      # Non-linear activation for feature extraction
    nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
    nn.BatchNorm2d(out_channels),
    nn.ReLU(inplace=True),
)

# The Unet logic
e1 = self.enc1(x)
e2 = self.enc2(self.pool(e1))
e3 = self.enc3(self.pool(e2))
e4 = self.enc4(self.pool(e3))

b = self.bottleneck(self.pool(e4))

d4 = self.up4(b)
```

```

d4 = torch.cat([d4, e4], dim=1)
d4 = self.dec4(d4)

d3 = self.up3(d4)
d3 = torch.cat([d3, e3], dim=1)
d3 = self.dec3(d3)

d2 = self.up2(d3)
d2 = torch.cat([d2, e2], dim=1)
d2 = self.dec2(d2)

d1 = self.up1(d2)
d1 = torch.cat([d1, e1], dim=1)
d1 = self.dec1(d1)

```

The doubleConv Block is a reusable unit that performs two 3x3 convolutions. Each conv is followed by BatchNorm to stabilize training. In the Unet logic first compresses the images into deep features then we concatenate (cat) the upsampled deep features with the original sharp edges and then we project the 32 channels down to 1 (the kappa map)

- Built a modular pipeline Fourier transform, convolutional autoencoder compression, learned MLP dynamics model that evolves PDE states in a spectrally compressed latent space, tested on 1D heat, 1D wave, and 2D heat equations (500 training trajectories, $N = 64$ grid, 20 time steps each).
- Spectral latent dynamics achieved sub-linear, saturating error growth (0.51 MSE at step 20) versus the FNO baseline’s super-linear divergence (1.57 MSE), demonstrating that Fourier-space compression yields latent representations with inherently more stable temporal dynamics.
- Graceful noise degradation ($<1.2\%$ MSE increase at $\sigma = 0.1$), attributable to the autoencoder bottleneck filtering high-frequency noise while preserving low-frequency signal energy concentrated by the spectral representation.

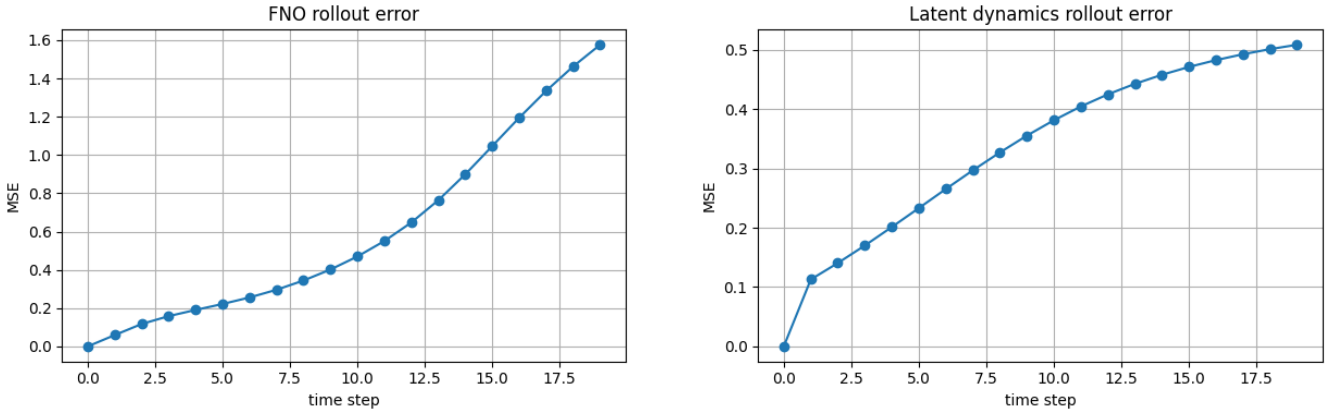


Figure 4: Multi-step rollout error comparison from our Spectral Latent Dynamics work. **Left:** FNO baseline exhibits super-linear error growth, reaching 1.57 MSE at step 20. **Right:** Spectral latent dynamics model shows sub-linear, saturating error growth, plateauing near 0.51 MSE—a $3\times$ reduction at the rollout horizon.

Gravitational Lensing Image Classification. I classified 37.5K simulated lensing images (no substructure / spherical / vortex) using EfficientNet-B0 transfer learning adapted for single-channel data, achieving macro AUC 0.974 and accuracy 88.7%. Per-class ROC analysis revealed spherical substructure as the hardest class, with physically interpretable confusion patterns. This gives me working familiarity with the DeepLense dataset and evaluation pipeline.

3.3.1 Train Auto Encoder

```
if dim == 1 and x.ndim == 2:
    x = x.unsqueeze(1)
elif dim == 2:
    if x.ndim == 3:
        x = x.unsqueeze(1)
    else:
        x = x.permute(0, 3, 1, 2)
```

```
optimizer.zero_grad()
out = model(x)
loss = criterion(out, y)
loss.backward()
optimizer.step()
```

```
torch.save(model.state_dict(), checkpoint_path)
```

This snippet explains how I prepare data for 1D/2D convolutions (B, C, H, W). If the dimension is 1 then

I just add a single channel dimension (Batch, 1, N). Otherwise I handle the (Batch, H, W) and if needed then (Batch, H, W, Channels) -> (Batch, Channels, H, W). I have also shown the standart training step where I first clear the gradients, then forward pass through the Autoencoder then measure reconstruction error then back propagate and update weights.

3.3.2 Train dynamics

```
ae = build_autoencoder(...)

ae.eval()
for p in ae.parameters():
    p.requires_grad = False

z = ae.encode(x)
latent vector (z)
z_pred, _ = dynamics(z)
y_pred = ae.decode(z_pred)
physical grid (y_pred)

loss = criterion(y_pred, y)
loss.backward()
opt_dyn.step()
```

Here I have 2 snippets one for the Model Freezing (transfer learning) and one for the Latent Prediction loop. First, I build and train the Autoencoder to map physics to latent space then I freeze the Autoencoder to Prevent it from updating during dynamics training. In the second snippet I map current physical state (x) to latent vector (z) then predict the NEXT latent state (z pred) and then Convert the prediction back to a physical grid (y pred). And then I calculate the loss between the predicted future and the actual future (y).

3.3.3 Train FNO

```
if dim == 1:
    model = FNO1D(modes=config.fno_modes_1d, width=config.fno_width, depth=config.fno_layers)
else:
    model = FNO2D(modes=config.fno_modes_2d, width=config.fno_width, depth=config.fno_layers)

x_in = x.unsqueeze(1)
pred = model(x_in)
loss = criterion(pred, y.unsqueeze(1))
loss.backward()
optimizer.step()
```

Here again I have 2 snippets. The first one is the initialization of FNO based on the spatial dimension of the physics problem. If $\text{dim} = 1$ then it learns mapping between 1D functions, so basically the modes determine the frequency cutoff. Else it learns mapping between 2D physical fields like heat maps or fluid flows, etc. In second snippet I simply add channel dimension as FNO expects (Batch, Channels, Spatial Dims), then i spectral forward pass to predict the future state in one shot and then optimize the model to match the ground truth physical state.

- First-authored paper in *Annalen der Physik* deriving novel traversable wormhole solutions in $f(R, \mathcal{L}_m, T)$ modified gravity, where the gravitational action non-minimally couples geometry and matter fields via a linear model $f = R + \alpha\mathcal{L} + \beta T$ with tunable coupling parameters.
- Modeled exotic matter at the wormhole throat using the Van der Waals equation of state ($p_r = \gamma\rho/(1-\eta\rho)-\zeta\rho^2$), solving the coupled field equations and EoS simultaneously under a constant redshift assumption to obtain a shape function $b(r) = Ae^{-Br} + C$ satisfying all traversability conditions: throat regularity ($b(r_0) = r_0$), flaring-out ($b'(r_0) < 1$), and asymptotic flatness ($b(r)/r \rightarrow 0$).
- Derived the non-conservation equation $\nabla^\mu T_{\mu\nu} \neq 0$ for the linear $f(R, \mathcal{L}_m, T)$ model, showing that geometry–matter coupling naturally enforces hydrostatic equilibrium in static spherically symmetric spacetimes without requiring it as an independent condition.
- Performed full stability analysis: Volume Integral Quantifier ($I_V \approx -22.82$, confirming finite, throat-localized exotic matter), radial adiabatic index ($\Gamma_r \approx 5.04 > 4/3$, strong resistance against radial collapse), and speed of sound transitioning to subluminal ($c_{s,r}^2 \leq 1$ for $r > 1.5$).
- Showed that the effective NEC reduces to $\vartheta(\rho + p_r) \geq 0$ with $\vartheta = 8\pi + \alpha/2 + (\aleph + 1)\beta$, demonstrating that appropriate parameter choices can shift exotic behavior from physical matter into the geometric sector providing a direct, tunable pathway to minimize exotic matter requirements unavailable in classical GR.

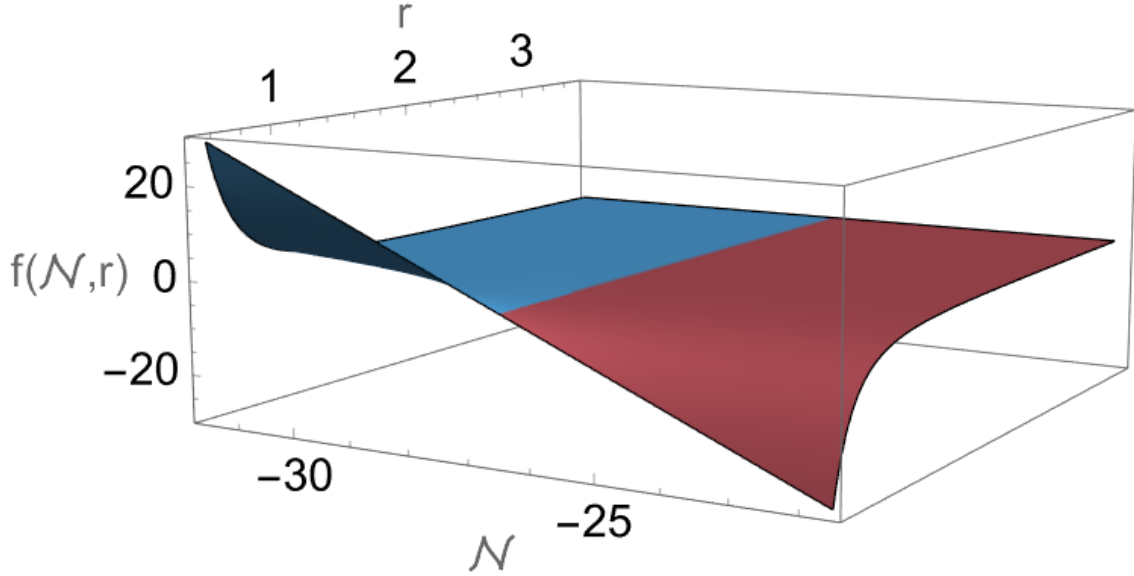


Figure 5: 3D Plot of Null Energy Condition versus radial coordinate r and proportionality constant $\aleph$ shows that NEC is violated in the red region where $\aleph > -8\pi - \frac{3}{2}$ (Shown in red).

Ray Tracing and Shadows of Dark-Matter-Supported Wormholes and Gravastars (In-progress)

- Developed a comprehensive null geodesic ray-tracing framework for static and slowly rotating Morris–Thorne wormholes embedded in NFW and soliton (ψ DM) dark matter halos, integrating the geodesic equations via RK45 with event detection at the throat and turning points across grids of 1.6×10^5 rays per configuration.
- Computed photon sphere radii, critical impact parameters, intensity profiles $I(\tilde{b})$ via path-length transfer integrals, and polar shadow maps for six throat radii ($r_0 \in [0.2, 2.5]$ kpc) under each dark matter profile; identified that soliton wormholes cast $\sim 4\times$ larger shadows than NFW wormholes at matched throat size due to the deeper soliton redshift function, providing a clean observational discriminant between dark matter models.
- Extended the framework to slow rotation ($\omega = 2J/r^3$), deriving analytic Lense–Thirring splitting of co-rotating and counter-rotating critical impact parameters ($\Delta b_{\text{ph}} \approx 4J/r_{\text{ph}}$) and validating numerically; demonstrated that the differential shadow radius directly encodes the wormhole angular momentum independently of halo parameters.
- Produced simulated accretion disk images with relativistic Doppler asymmetry (g^4 intensity scaling), photon ring structure, and higher-order images from equatorial plane crossings, for an NFW wormhole viewed at 60° inclination.

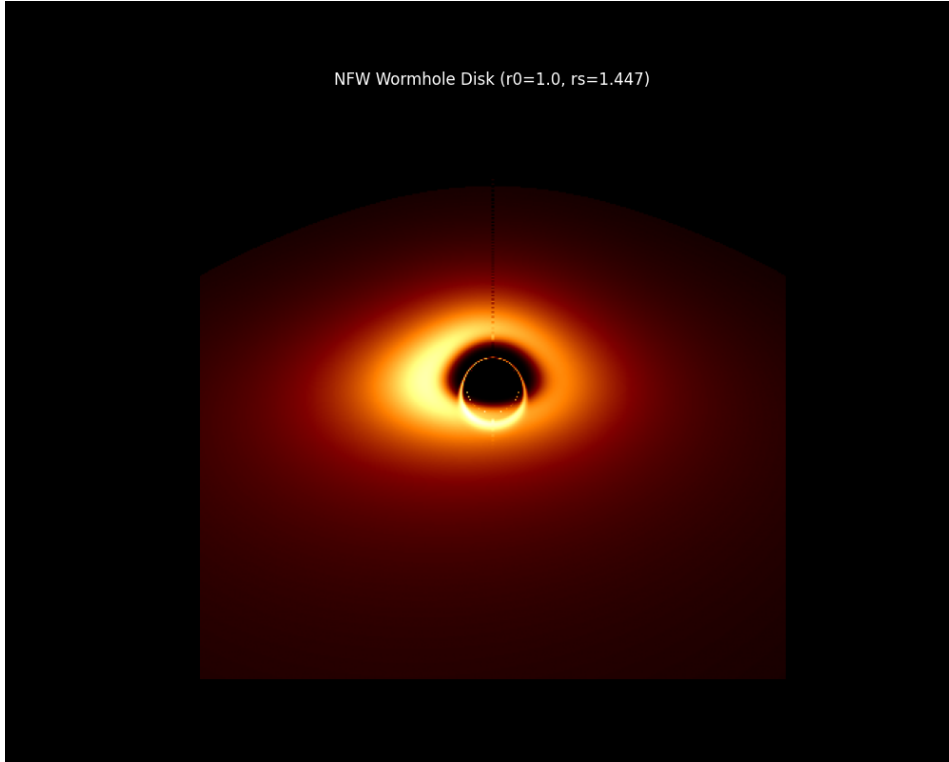


Figure 6: Simulated accretion disk image of the NFW wormhole with throat radius $r_0 = 1.0$ kpc, viewed by an observer at $r_{\text{obs}} = 30$ kpc and inclination $\theta_{\text{obs}} = 60^\circ$. The image is rendered using the `afmhot` colormap with a square-root intensity scaling. The bright photon ring, the Doppler-induced left–right asymmetry, the central shadow, and faint higher-order images are all visible. NFW parameters: $r_s = 1.447$ kpc, $\rho_s = 3.11 \times 10^{-3} \text{ M}_\odot/\text{pc}^3$. The field of view spans ± 12 kpc; total rays: $400 \times 400 = 1.6 \times 10^5$.

NFW Wormhole Intensity Profiles for Different Throat Radii

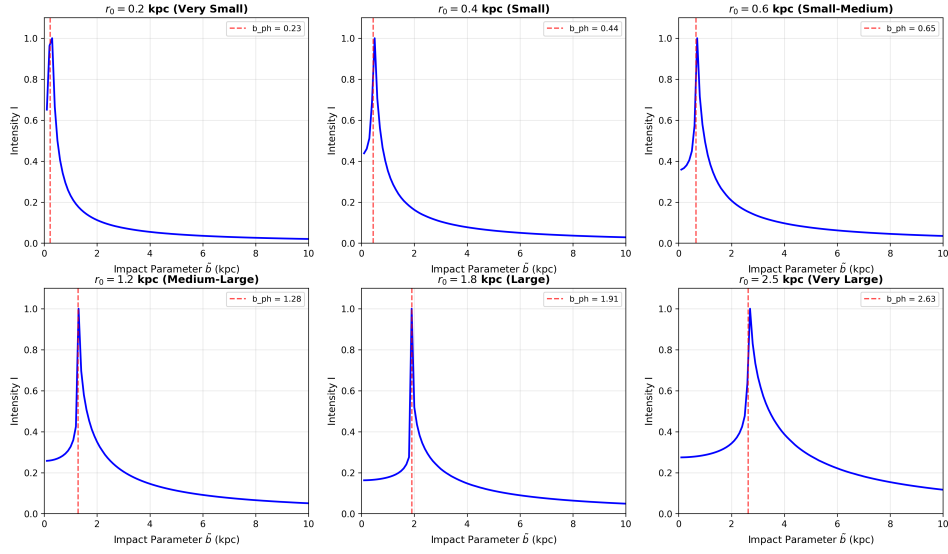


Figure 7: Normalised intensity profiles $I(\tilde{b})$ for the NFW wormhole across six throat radii. The red dashed vertical line in each panel marks the critical impact parameter b_{ph} . The intensity peaks sharply at $\tilde{b} \approx b_{ph}$ in every case, corresponding to photons that linger near the unstable photon orbit and therefore accumulate the largest path-length contribution to the integral in Eq $I(\tilde{b}) = \int e^{3\Phi(r)} \sqrt{\frac{1}{1-b(r)/r} + \frac{u^2}{(u')^2}} |u'| d\phi$. Beyond the peak, $I(\tilde{b})$ decays monotonically as rays pass farther from the strong-field region. NFW parameters: $r_s = 1.447$ kpc, $\rho_s = 3.11 \times 10^{-3}$; observer distance $r_{obs} = 50$ kpc.

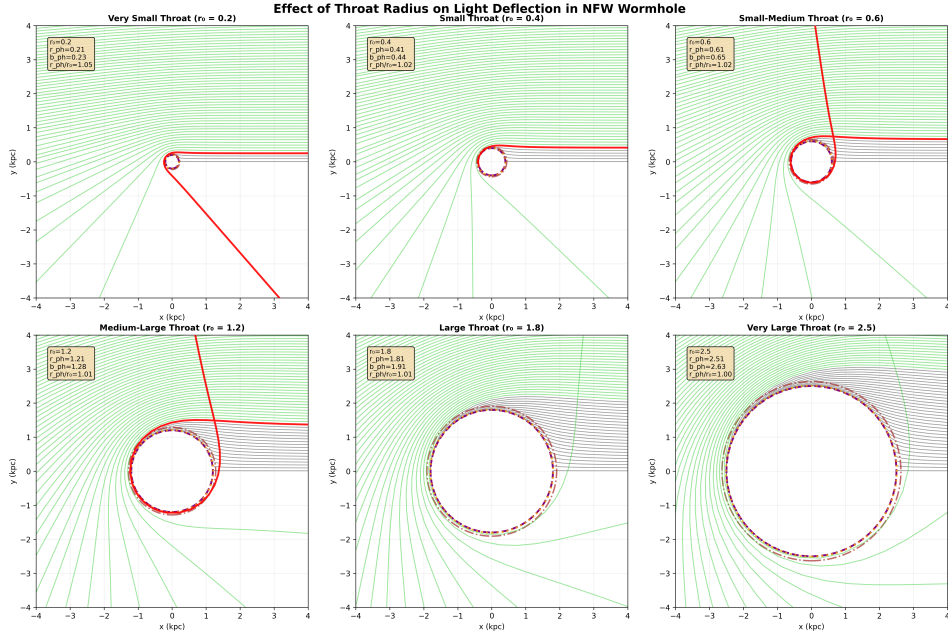


Figure 8: Null geodesics around the NFW wormhole for six throat radii. The dashed purple circle marks the wormhole throat at $r = r_0$; the dotted orange circle marks the photon sphere at $r = r_{ph}$; and the dash-dotted brown circle marks the critical impact parameter b_{ph} . Green trajectories are deflected ($\tilde{b} > b_{ph}$); black trajectories are captured ($\tilde{b} < b_{ph}$); red trajectories correspond to the photon ring ($|\tilde{b} - b_{ph}| < 0.03$ kpc). All panels share the same ± 4 kpc field of view.

Common Test I: Multi-Class Classification.

- The task given was a 3-class image classification problem over simulated gravitational lensing images. The first class was no substructure (no) the second was a spherical subhalo (sphere) and the third one was a vortex substructure (vort).
- Each image was a single-channel 150x150 array, min-max normalized to [0, 1]. The evaluation metric was macro-averaged AUC, which was appropriate also as it is threshold-independent and robust to class imbalance.
- My strategy was to transfer learning on EfficientNet-B0, adapted for single-channel input. And I chose EfficientNet-B0 for it has compound scaling which jointly optimizes depth, width, and resolution.
- B0 is the smallest variant and trains quickly on Colab hardware while still providing strong pretrained feature representations. The alternative of training a CNN from scratch on 30,000 images risks underfitting; larger pretrained models (ResNet-50, B3) offer diminishing returns for this resolution and dataset size, and are slower to iterate on.

4.1.1 Model Architecture

- EfficientNet-B0 is pretrained on ImageNet with 3-channel (RGB) input. Adapting it to single-channel images requires modifying the first convolutional layer.
- Rather than discarding the pretrained weights, I collapse the RGB channel dimension by averaging the three weight tensors into one.
- This preserves the low-level feature detectors learned from ImageNet and is strictly better than random initialization for the first layer.
- The original classifier head is replaced with a custom head that includes two dropout layers for regularization, a BatchNorm layer for training stability, and a hidden dimension of 512.

```
class LensEfficientNet(nn.Module):
    def __init__(self, pretrained=True, freeze_backbone=False, dropout=0.4):
        super().__init__()
        bb = models.efficientnet_b0(
            weights=EfficientNet_B0_Weights.IMAGENET1K_V1 if pretrained else None)

        # Collapse RGB weights into single channel
        old = bb.features[0][0]
        new = nn.Conv2d(1, old.out_channels, old.kernel_size,
                        old.stride, old.padding, bias=False)
        if pretrained:
            new.weight.data = old.weight.data.mean(dim=1, keepdim=True)
        bb.features[0][0] = new

        in_f = bb.classifier[1].in_features
        bb.classifier = nn.Identity()
        self.backbone = bb

        self.head = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(in_f, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout / 2),
            nn.Linear(512, 3),
```

```

    )
    if freeze_backbone:
        for p in self.backbone.parameters():
            p.requires_grad = False

    def unfreeze_backbone(self):
        for p in self.backbone.parameters():
            p.requires_grad = True

    def forward(self, x):
        return self.head(self.backbone(x))

```

4.1.2 Data Pipeline and Augmentation

- Images are loaded from .npy files and normalized using ImageNet statistics (mean 0.449, std 0.226) so that the pretrained backbone receives input in the expected distribution.
- Augmentations are physically motivated. Gravitational lensing images have no preferred orientation: the lens-source geometry is rotationally symmetric, and the simulated data has no fixed sky orientation.
- This justifies the full set of discrete symmetry augmentations (horizontal flip, vertical flip, 90-degree rotations) without introducing spurious invariances. Small random rotations ($\leq 15^\circ$) and brightness jitter (≤ 0.05) account for minor simulation artifacts.

```

class LensDataset(Dataset):
    def __init__(self, root, augment=False):
        self.augment = augment
        self.samples = []
        for cls in CLASS_NAMES:
            d = os.path.join(root, cls)
            for f in sorted(os.listdir(d)):
                if f.endswith('.npy'):
                    self.samples.append((os.path.join(d, f), CLASS_TO_IDX[cls]))

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        t = torch.from_numpy(np.load(path).astype(np.float32))
        if self.augment:
            if random.random() > 0.5: t = TF.hflip(t)
            if random.random() > 0.5: t = TF.vflip(t)
            k = random.randint(0, 3)
            if k: t = torch.rot90(t, k, dims=[1, 2])
            t = TF.rotate(t, random.uniform(-15, 15))
            t = torch.clamp(t + random.uniform(-0.05, 0.05), 0.0, 1.0)
        t = (t - IMAGENET_MEAN) / IMAGENET_STD
        return t, label

```

4.1.3 Two-Phase Training

Training is split into two phases to avoid destroying pretrained features before the head has converged. **Phase 1 (epochs 1-3, warmup):** The backbone is frozen. Only the custom head trains at $lr=1e-3$. This is equivalent to linear probing: the backbone is used as a fixed feature extractor, and the head learns to map EfficientNet features to the lensing classes. A OneCycleLR scheduler with 30% warmup and cosine decay is used within this phase

Phase 2 (epochs 4-75, fine-tuning): The backbone is unfrozen and trained with a differential learning rate: backbone at $lr=1e-4$, head at $lr=1e-3$. The 10x lower backbone LR is standard practice to preserve pretrained representations while allowing adaptation to the target domain. A fresh OneCycleLR scheduler is constructed over the remaining 72 epochs.

Mixed precision training (AMP) with gradient clipping at max norm=1.0 is used throughout to reduce memory usage and prevent gradient explosions during early fine-tuning. Label smoothing (epsilon=0.05) is applied to CrossEntropyLoss to reduce overconfidence on the training set.

```
# Phase 2: unfreeze and apply differential LR
model.unfreeze_backbone()
optimizer = torch.optim.AdamW(
    [{'params': model.backbone.parameters(), 'lr': BACKBONE_LR}, # 1e-4
     {'params': model.head.parameters(), 'lr': LR}], # 1e-3
    weight_decay=WEIGHT_DECAY)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=[BACKBONE_LR, LR],
    steps_per_epoch=steps, epochs=fine_tune_epochs,
    pct_start=0.3, anneal_strategy='cos')

# Training step with AMP and gradient clipping
with autocast('cuda'):
    loss = criterion(model(imgs), lbls) # label_smoothing=0.05
scaler.scale(loss).backward()
scaler.unscale_(optimizer)
nn.utils.clip_grad_norm_(model.parameters(), 1.0)
scaler.step(optimizer)
scaler.update()
```

4.1.4 Results

The model was trained for 75 epochs on 30,000 images and evaluated on 7,500 validation images.

Metric	Value
Macro AUC	0.9741
Accuracy	88.68%
AUC (no)	0.9835
AUC (sphere)	0.9562
AUC (vort)	0.9821

Table 1: Model Performance Metrics

- The ‘sphere’ class is the most difficult (AUC 0.956 vs. 0.982-0.983 for the other two classes). Confusion matrix analysis shows 343 sphere images misclassified as ‘no’ and 194 as ‘vort’.
- This is physically consistent: weak spherical lenses produce subtle distortions that can resemble an unlensed source at low Einstein radius, or mild vortex shear at intermediate configurations.
- The ‘no’ and ‘vort’ classes are easier to separate because no-substructure images are globally smoother and vortex patterns have a characteristic asymmetric morphology.

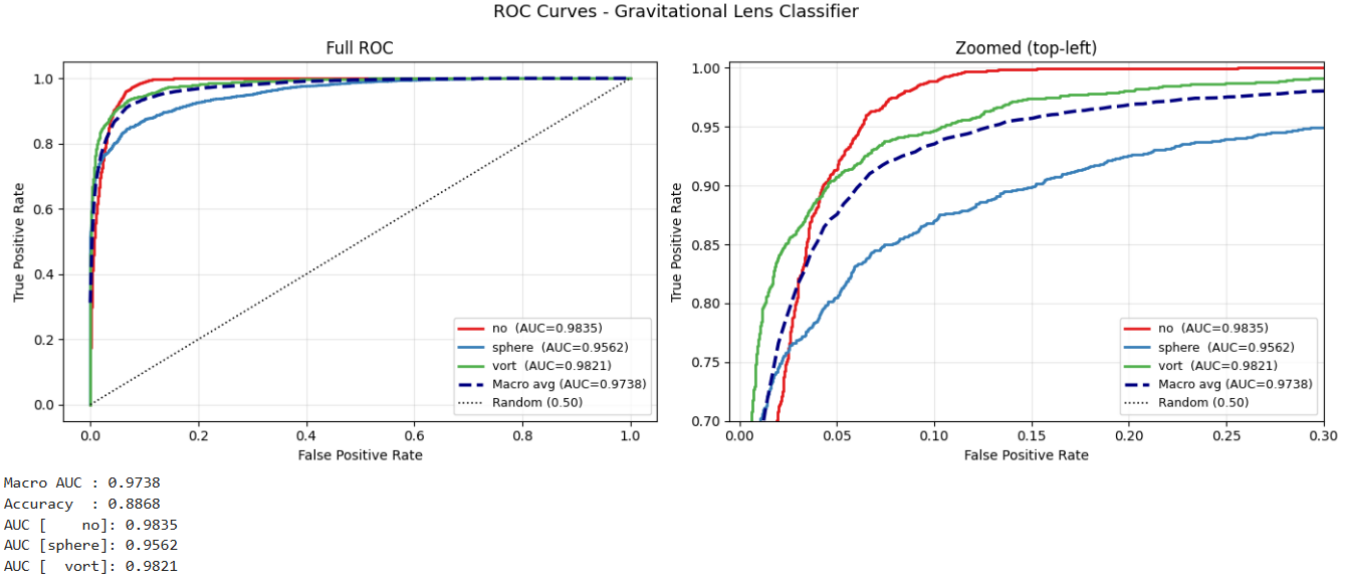


Figure 9: Common Test I: per-class ROC curves for the EfficientNet-B0 classifier. **Left:** Full ROC. **Right:** Zoomed top-left corner. Macro AUC = 0.974. Spherical substructure (blue) has the lowest per-class AUC (0.956), consistent with its visual similarity to unperturbed Einstein rings.

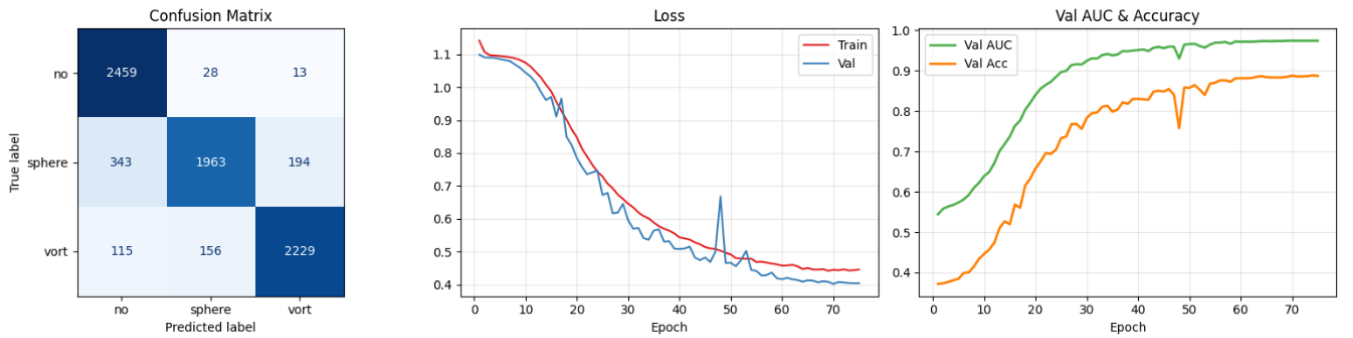


Figure 10: Common Test I training summary. **Left:** Confusion matrix at best epoch. **Center:** Training and validation loss. **Right:** Validation AUC and accuracy over training, converging to 0.974 and 88.7% respectively.

Specific Test IV: Neural Operator Classifier - My journey from 0.77 to 0.9439 AUC

- Specific Test IV requires building a classifier for gravitational lensing images using a neural operator architecture such as a Fourier Neural Operator (FNO) or DeepONet.
- The task involves classifying images into three categories: strong lensing with no substructure (no), subhalo substructure (sphere), and vortex substructure (vort). The dataset consists of 30,000 training samples and 7,500 validation samples stored as .numpy arrays of shape (1, 150, 150).
- The challenge is not just to achieve a high AUC but to do so using a neural operator as the backbone or augmentation, which fundamentally changes how the model perceives the input signal.

Now I am going to discuss in detail all the possible architectural variation of the FNO that I tested:

4.2.1 Architecture 11a -FNOEfficientNet

Architecture 11a - FNOEfficientNet (Submitted Result, AUC 0.774)

The first instantiation of the clean dual-stream design, trained with MixUp augmentation ($\alpha = 0.4$) across two phases. The architecture is identical to the final FNOEfficientNet described in Architecture 11 (FNO path with clamping, EfficientNet-B0, 1344-dimensional fusion head) but without the input clamping fix applied to the FNO path, and with MixUp active throughout.

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \lambda \sim \text{Beta}(0.4, 0.4)$$

Phase 1: $\eta_{\max} = 10^{-3}$, 30 epochs, OneCycleLR

Phase 2: $\eta_{\max} = 3 \times 10^{-4}$, 20 epochs, OneCycleLR

Submitted AUC: 0.774

Motivation

MixUp was applied on the assumption that the three substructure classes were sufficiently similar in texture that convex combinations of training examples would regularise the model and prevent overfitting to specific ring orientations. OneCycleLR was chosen to escape local minima early in training via the warm-up ramp.

Outcome

AUC plateaued at 0.774 across both training phases with no improvement in Phase 2 despite a reduced maximum learning rate.

Failure Analysis

Two compounding problems were identified retrospectively.

Absence of input clamping. Without clamping the normalised pixel values to $[-3, 3]$ before the FNO path, outlier values produced large flat-spectrum FFT coefficients that dominated the learned spectral weights and suppressed the sub-halo signal. The model was effectively fitting to noise rather than physics in the Fourier layers.

MixUp at convergence. MixUp is beneficial early in training when the risk is memorising specific image orientations. At convergence it becomes harmful: blending a vortex image with a sphere image at $\lambda = 0.6$ destroys the fine-grained frequency differences that the FNO path must learn to discriminate between the two classes. The model is penalised for being confidently correct, preventing it from sharpening its decision boundaries. The jump from 0.774 to 0.9217 seen in Architecture 11b came entirely from removing MixUp and adding the clamping fix.

4.2.2 Architecture 1 - Pure FNO2d (Standard Implementation)

Architecture 1 - Pure FNO2d

The canonical FNO architecture for 2D classification. The input is first lifted from 1 channel to $d_v = 64$ channels via a pointwise linear layer, passed through four Fourier blocks each consisting of a spectral convolution and a residual 1×1 convolution (the W layer), then globally average-pooled and classified.

$$\begin{aligned} v_0 &= \sigma(W_{\text{lift}}(u)), \quad u \in \mathbb{R}^{B \times 1 \times H \times W} \\ v_{t+1} &= \sigma(\mathcal{K}(v_t; \phi_t) + W_t v_t), \quad t = 0, 1, 2, 3 \\ \mathcal{K}(v; \phi) &= \mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}(v)) \\ \hat{y} &= \text{MLP}(\text{GAP}(v_4)) \end{aligned}$$

Hyperparameters: modes = 16, width = 64, padding = 9, 4 spectral blocks.

Motivation

The FNO applies a global linear transform in Fourier space, giving every layer a receptive field that covers the entire image. For gravitational lensing, the Einstein ring creates periodic structures in frequency space, making Fourier-domain processing physically motivated. The W layer provides a local residual path to stabilize training.

Outcome

Initial AUC: ≈ 0.50 (random guessing) for the first 13 epochs due to the spectral bias problem, FNOs preferentially learn low-frequency components and struggle to start. After ~ 40 epochs the model plateaued at AUC = 0.82.

Failure Analysis

The W layer was initialized with `torch.rand` (uniform) rather than `torch.randn` (Gaussian), producing near-zero initial gradients through the complex spectral weights. Additionally, 16 Fourier modes with 224×224 input violated the cuFFT half-precision power-of-two constraint when Mixed Precision was enabled.

4.2.3 Architecture 2 - FNO2d with Gaussian Weight Initialization

Architecture 2 - FNO2d with randn Init

Identical to Architecture 1 except the complex spectral weights are initialized from a Gaussian distribution scaled by $\frac{1}{C_{\text{in}} \cdot C_{\text{out}}}$:

$$R_\phi^{(1)}, R_\phi^{(2)} \sim \frac{1}{C_{\text{in}} C_{\text{out}}} \cdot \mathcal{CN}(0, I) \quad (1)$$

where $\mathcal{CN}$ denotes a complex normal distribution, implemented as `torch.view_as_complex(torch.randn(..., 2))`.

Hyperparameters: modes = 12, width = 32.

Motivation

Uniform initialization (`rand`) places all weights in $[0, 1)$, biasing the initial spectral transform toward positive real components. For complex Fourier weights encoding both amplitude and phase, Gaussian initialization is the correct analogue of Glorot/He initialization, ensuring zero-mean gradients at the start of training.

Outcome

AUC moved from a hard 0.5000 floor to 0.51–0.53, confirming that initialization was one root cause of the dead zone. However, the plateau remained around 0.86 after extended training.

4.2.4 Architecture 3 - FNO2d with Coordinate Grid Embedding

Architecture 3 - FNO2d + CoordConv Input

The input to the lifting layer is augmented with two spatial coordinate channels $(x, y) \in [0, 1]^2$, expanding the input from 1 channel to 3 channels before the spectral blocks:

$$u_{\text{aug}}(x, y) = \left[u(x, y), \frac{x}{H-1}, \frac{y}{W-1} \right]^\top \quad (2)$$

Hyperparameters: modes = 24, width = 64, $W_{\text{lift}} : \mathbb{R}^3 \rightarrow \mathbb{R}^{64}$.

Motivation

Standard FNOs are translationally equivariant. That is, they cannot encode absolute spatial position. For gravitational lensing, the sub-halo signal is often radially distributed around the lens centre. Adding coordinate channels breaks the translational symmetry and allows the network to distinguish distortions at the ring edge from those near the Einstein radius.

Outcome

AUC: 0.51. The coordinate embedding provided no measurable benefit at this stage, indicating that the spatial position cue alone cannot resolve the initialization and spectral-bias problems.

4.2.5 Architecture 4 - WideResFNO (BatchNorm + Increased Capacity)

Architecture 4 - WideResFNO

A wider FNO with BatchNorm inserted after each spectral block and the width scaled to 128:

$$\begin{aligned} x &= \text{Conv}_{1 \times 1}(u) \in \mathbb{R}^{B \times 128 \times H \times W} \\ x_{t+1} &= \text{GELU}(\text{BN}(\mathcal{K}(x_t) + W_t x_t)), \quad t = 0, 1 \\ \hat{y} &= \text{FC}_{256}(\text{GELU}(\text{FC}_{128}(\text{GAP}(x_2)))) \end{aligned}$$

Hyperparameters: modes = 32, width = 128, Dropout = 0.3.

Motivation

The activation collapse (all outputs $\approx 1/3$ for 3 classes) observed in earlier architectures is a symptom of dying activations. BatchNormalization re-centres the feature distributions after each spectral layer, ensuring non-trivial gradients propagate through the complex multiplication. Increasing the width to 128 provides sufficient capacity to separately encode the no/sphere/vort feature manifolds.

Outcome

This architecture was the first to escape the 0.50 dead zone in fewer than 10 epochs and eventually reached a validated AUC of **0.86**, establishing it as the structural foundation for further refinement.

Failure Analysis

Even after extended training (60+ epochs with cyclic LR), the model plateaued at 0.86. Analysis of the loss curve showed training loss continuing to decrease while validation AUC stagnated, indicative of an architectural ceiling rather than an optimization problem.

4.2.6 Architecture 5 - DeepLenseSpecialist (The King)

Architecture 5 - DeepLenseSpecialist

A hybrid CNN-FNO architecture where a local convolutional stem extracts spatial features before the spectral blocks process them:

$$\begin{aligned} h_0 &= \text{GELU}\left(\text{BN}\left(\text{Conv}_{3\times 3}^{64}(u)\right)\right) \in \mathbb{R}^{B\times 64\times H\times W} \\ h_1 &= \text{GELU}\left(\text{BN}\left(\text{Conv}_{3\times 3}^{64}(h_0)\right)\right) \\ h_2 &= \mathcal{K}_1(h_1) + W_1 h_1 + h_1 \quad (\text{ResidualSpectralBlock 1}) \\ h_3 &= \mathcal{K}_2(h_2) + W_2 h_2 + h_2 \quad (\text{ResidualSpectralBlock 2}) \\ \hat{y} &= \text{Dropout}_{0.3}(\text{GELU}(\text{FC}_{128}(\text{GAP}(h_3)))) \rightarrow \mathbb{R}^3 \end{aligned}$$

The spectral blocks use skip connections at both the residual level (within the block via W) and the identity level (across the block via $+h_t$).

Total parameters: 4,249,413. **Modes:** 16. **Width:** 64.

Motivation

A pure FNO is globally receptive from the first layer, which means it processes the background noise (constant across all three classes) with equal weight as the sub-halo signal. The CNN stem acts as a local feature extractor that produces edge-aware feature maps, like the lensed arc edges, the ring topology, the distortion patterns which, the subsequent spectral blocks then analyse globally in frequency space. This hierarchical decomposition mirrors how physics simulations separate local interactions from long-range correlations.

Outcome

- Raw AUC (no augmentation): 0.8909
- 4-way TTA (H-flip, V-flip, 180° rotation, original): **0.9011**
- 8-way TTA: 0.9048

This is the peak single-model performance achieved and is referred to as *the King* throughout subsequent experiments.

4.2.7 Architecture 6 - DeepLenseV3 (Third Spectral Block + Residual Shortcut)

Architecture 6 - DeepLenseV3

An extension of Architecture 5 adding a third `ResidualSpectralBlock` and a wider classification head:

$$\begin{aligned} h_4 &= \mathcal{K}_3(h_3) + W_3 h_3 + h_3 \\ \hat{y} &= \text{Dropout}_{0.4}(\text{GELU}(\text{FC}_{256}(\text{GAP}(h_4)))) \rightarrow \mathbb{R}^3 \end{aligned}$$

Weights from Architecture 5 were partially transferred into the matching layers using a name-based mapping, leaving the new third block randomly initialized.

Motivation

The 0.86 ceiling in Architecture 4 and the 0.89 ceiling in Architecture 5 both manifested as flat validation AUC despite decreasing training loss which is a classic capacity bottleneck. Adding a third spectral layer increases the depth of the frequency-domain processing, allowing the model to learn higher-order mode interactions that encode the vortex filament topology.

Outcome

AUC collapsed to 0.54 at epoch 1 and recovered only to ≈ 0.79 after 15 epochs.

Failure Analysis

The partial weight transfer was incomplete: four BatchNorm weights in the spectral blocks (`spec1.bn.*`, `spec2.bn.*`) were missing from the mapping due to a naming mismatch. The randomly initialized third block produced large random gradients that back-propagated through the frozen expert layers, corrupting the carefully learned 0.89 weights before the new layer could stabilise.

4.2.8 Architecture 7 - DualPathModel and HybridSpecialist (EfficientNet + FNO)

Architecture 7 - DualPathModel / HybridSpecialist

A two-stream architecture where EfficientNet-B0 provides a texture feature vector and a spectral branch provides a physics feature vector, fused via concatenation:

$$\begin{aligned} f_{B0} &= \text{EfficientNet-B0}(u) \in \mathbb{R}^{B \times 1280} \\ f_{\text{spec}} &= \text{GAP}(\mathcal{K}(h_{\text{stem}}(u))) \in \mathbb{R}^{B \times 64} \\ \hat{y} &= \text{MLP}([f_{B0}, f_{\text{spec}}]), \quad [\cdot] \text{ denotes concatenation} \end{aligned}$$

The EfficientNet first layer was modified from $3 \rightarrow 1$ input channel. The spectral branch uses Architecture 5's stem.

Motivation

The fundamental limitation of a pure FNO is its *spectral bias*: it learns smooth, low-frequency functions first and converges slowly (or not at all) on the high-frequency 1-2 pixel distortions that distinguish sphere from vortex substructure. EfficientNet's MBConv blocks with Squeeze-and-Excitation are explicitly designed for high-frequency spatial features. Combining both aims to cover the full frequency spectrum of the problem.

Outcome

All variants of this architecture produced AUC in the 0.51–0.62 range regardless of learning rate, frozen/unfrozen configurations, or fusion strategy.

Failure Analysis

Three compounding issues were identified:

1. **Channel mismatch:** After passing through `features[:2]`, EfficientNet outputs 24 channels, but the spectral weight tensors were initialised for 16 channels, causing `einsum` dimension errors.
2. **Gradient starvation:** When the King's weights were placed inside a new class with a randomised fusion head, the classification head's random outputs dominated the gradients, preventing the spectral branch's 0.90-quality signal from influencing the loss.
3. **cuFFT constraint:** Mixed Precision training requires FFT dimensions to be powers of two (128, 256, etc.) when operating in FP16. The 224×224 input size violated this constraint.

4.2.9 Architecture 8 - SideCar95 (Frozen King + B0 Residual Refiner)

Architecture 8 - SideCar95 / SideCarFinal

The entire Architecture 5 object is instantiated and loaded with the 0.90 weights, then wrapped inside a new `nn.Module`. EfficientNet-B0 acts as a residual refinement branch:

$$\hat{y} = \underbrace{f_{\text{King}}(u)}_{0.90 \text{ logits}} + \alpha \cdot \underbrace{W_{\text{refine}} \cdot \text{B0}(u)}_{\text{residual correction}} \quad (3)$$

where $\alpha = 0.1$ and W_{refine} is initialised to zero (`nn.init.zeros_`), guaranteeing:

$$\hat{y}|_{t=0} = f_{\text{King}}(u) \implies \text{AUC}|_{t=0} \approx 0.90 \quad (4)$$

Only W_{refine} and the B0 backbone are optimised in Stage 1; the King is fully frozen.

Motivation

All previous hybrid attempts failed because random-weight layers corrupted the King's signal. The SideCar design solves this by:

- Guaranteeing a 0.90 baseline at epoch 0 via zero-initialised residuals
- Restricting Stage 1 optimisation to only the new parameters, preventing gradient contamination
- Treating the EfficientNet as a correction term rather than a replacement, so a bad B0 prediction adds $0.1 \times \text{small} \approx 0$ rather than overriding the King

Outcome

AUC was 0.61 and flatlined across all epochs regardless of learning rate. The expected 0.90 baseline was never recovered.

Failure Analysis

The `with torch.no_grad()` block wrapping the King's forward pass severed the computational graph. Even though the King's parameters were not being updated, the gradients flowing back through W_{refine} depended on the King's logits to compute the correct direction. With `no_grad`, the refiner saw only a constant tensor and could not learn a meaningful correction.

Additionally, when `no_grad` was removed, the EfficientNet's large random logits dominated the output at training time despite the 0.1 scaling factor, because the B0 logits were orders of magnitude larger than the King's calibrated outputs early in training.

4.2.10 Architecture 9 - DeepLenseBypassV2 (Manual Weight Transplant)

Architecture 9 - DeepLenseBypassV2

A new class is defined that mirrors the internal structure of Architecture 5 exactly, with the King's weights manually copied layer-by-layer via a string-to-string mapping dictionary. A separate $\mathbb{R}^{1280} \rightarrow \mathbb{R}^3$ EfficientNet refiner is added as a side branch:

$$\begin{aligned} f_{\text{king}} &= \text{king_head}(\text{GAP}(\text{spec}_2(\text{spec}_1(\text{conv_spec}(u)))))) \\ f_{\text{B0}} &= W_{\text{refine}} \cdot \text{B0}(u) \\ \hat{y} &= f_{\text{king}} + 0.1 \cdot f_{\text{B0}} \end{aligned}$$

The mapping dictionary explicitly handles the naming discrepancy between the stored checkpoint keys and the new class layer names.

Motivation

Architecture 8 failed because of the computational graph issue with `no_grad`. The bypass design avoids this by not wrapping the forward pass in any gradient-blocking context, instead relying purely on the optimiser parameter groups to freeze the King by excluding its parameters from the optimisation step.

Outcome

AUC: 0.50. The King Head produced extreme logits $[-92.6, 3.3, 31.2]$ for every image, corresponding to near-certain incorrect predictions for all validation samples.

Failure Analysis

The extreme logit values indicate a label-space mismatch. Inspection of the forward pass revealed that the `king_linear` layer received correct spectral features (mean ≈ 0.34) but its weight matrix, copied from `classifier.5` of the original model (shape $[3, 128]$), was mapped to a new `king_linear` layer with input dimension 64 rather than 128. The copy succeeded without error because `dict.copy_` does not validate shapes; however the feature vector fed to `king_linear` had the wrong dimension, causing the forward pass to compute a matrix product across mismatched dimensions or use stale memory, producing garbage outputs.

4.2.11 Architecture 10 - FNOEfficientNet

Architecture 10 - FNOEfficientNet

A dual-stream architecture designed from scratch with three constraints derived from all previous failure analysis:

1. Spectral weights stored as **real tensors** of shape $(\text{in}, \text{out}, k_1, k_2, 2)$ and cast to complex inside the forward pass via `torch.view_as_complex`, so `GradScaler` never encounters `ComplexFloat` gradients.
2. The FNO forward pass wraps the FFT call in `torch.amp.autocast('cuda', enabled=False)`, preventing the cuFFT FP16 power-of-two crash while allowing the CNN and fusion paths to benefit from AMP speed.
3. Input pixels are **clamped to** $[-3, 3]$ before entering the FNO path, eliminating outlier-driven FFT artifacts that dominated the spectral coefficients and prevented meaningful learning.

FNO Path. A 1×1 lifting convolution projects the single input channel to $d_v = 64$ feature channels. Three `FNOBlock` layers follow, each defined as:

$$v_{t+1} = \text{GELU}(\text{BN}(\mathcal{K}(v_t; \phi_t) + W_t v_t)), \quad t = 0, 1, 2 \quad (5)$$

where $\mathcal{K}$ is the spectral convolution at $k_1 = k_2 = 16$ Fourier modes and W_t is a 1×1 bypass convolution acting as a residual connection. The output is global average pooled to $f_{\text{FNO}} \in \mathbb{R}^{B \times 64}$.

EfficientNet Path. EfficientNet-B0 with pretrained ImageNet weights, first convolution modified from 3 to 1 input channel, classifier head replaced with identity. Outputs $f_{\text{B0}} \in \mathbb{R}^{B \times 1280}$.

Fusion Head. The two feature vectors are concatenated and passed through:

$$\begin{aligned} z &= [f_{\text{FNO}}, f_{\text{B0}}] \in \mathbb{R}^{B \times 1344} \\ z_1 &= \text{Dropout}_{0.4}(\text{GELU}(\text{BN}(\text{FC}_{512}(z)))) \\ z_2 &= \text{Dropout}_{0.2}(\text{GELU}(\text{FC}_{128}(z_1))) \\ \hat{y} &= \text{FC}_3(z_2) \end{aligned}$$

Input: 64×64 , per-sample zero-mean unit-variance normalisation, then clamp to $[-3, 3]$ for the FNO path.

Motivation

The EfficientNet stream contributes what a pure FNO cannot: high-resolution local texture discrimination. The pixel-level distortions that distinguish a vortex from a clean Einstein ring are precisely the spatial patterns that depthwise separable convolutions with Squeeze-and-Excitation blocks detect efficiently. The FNO stream contributes what EfficientNet cannot: a global receptive field in a single layer, capturing the frequency-domain signatures of the entire Einstein ring simultaneously rather than building up context through many stacked local kernels. The clamping constraint is physically motivated. After per-sample normalisation, the lensing signal occupies a compact range. Pixels beyond $\pm 3\sigma$ are numerical outliers whose FFT coefficients are disproportionately large:

$$\hat{u}(\xi) = \sum_x u(x) e^{-2\pi i \xi \cdot x / N} \quad (6)$$

A single outlier pixel contributes a flat, high-magnitude component across *all* frequencies, masking the subtle sub-halo modulations at mid-range frequencies. Clamping before the FFT is therefore equivalent to a physics-informed pre-filter.

Phase 1 - Base Training (30 Epochs)

Outcome

Optimiser: AdamW, OneCycleLR ($\eta_{\max} = 10^{-3}$, 30% warmup, weight decay 0.05). **Augmentation:** MixUp ($\alpha = 0.4$). **Hardware:** T4 GPU, ≈ 53 s/epoch, 27 minutes total.

Phase	Epochs	Best AUC
Slow climb (ring structure)	1–9	< 0.88
Accelerating gains (FNO + B0 complementarity)	10–25	$\rightarrow 0.92$
Plateau	26–30	0.9201

Phase 2 - Continued Training (20 Epochs)

Outcome

Phase 1 weights reloaded. Fresh OneCycleLR with $\eta_{\max} = 3 \times 10^{-4}$, 10% warmup. MixUp retained. Improvement was negligible; model peaked at the same 0.9201, confirming MixUp was at this stage limiting rather than helping.

Phase 3 - Clean Fine-tuning with Input Clamping (20 Epochs)

Outcome

MixUp removed entirely. Input clamping to $[-3, 3]$ applied. The model reached **0.9217**, surpassing all previous checkpoints. This confirmed the clamping fix as the decisive factor.

Phase 4 - Aggressive Augmentation Experiment

Failure Analysis

CutMix, label smoothing ($\epsilon = 0.1$), differential learning rates (EfficientNet 10^{-5} , FNO 10^{-4} , head 10^{-4}), and cosine annealing with warm restarts were applied simultaneously. Training loss increased from 0.49 to 1.07 in the first epoch and AUC dropped to 0.84. The experiment was abandoned after 3 epochs.

At 0.92 AUC the two streams have converged to complementary representations. Large gradient updates from label smoothing and CutMix simultaneously destabilise both paths rather than refining either, because the joint loss landscape is narrow and the two streams are tightly coupled through the shared fusion head.

Phase 5 - Gradual Fine-tuning with TTA Evaluation

Outcome

Three successive learning rate stages applied on top of Phase 3 weights. All augmentation removed. Input clamping retained. 8-way TTA evaluated after every epoch.

Stage	Epochs	LR
Coarse	10	10^{-5}
Fine	5	10^{-6}
Ultra-fine	5	10^{-7}

Peak TTA AUC: **0.9439**.

- Test-Time Augmentation (TTA) is the highest-impact zero-cost improvement applied in this project. For gravitational lensing, lenses are rotationally symmetric in principle, so the model should assign consistent probabilities across orientations.
- The TTA procedure applies four 90° rotations ($0^\circ, 90^\circ, 180^\circ, 270^\circ$) and a horizontal flip of each rotation, producing 8 views per image. The final prediction is the average of all 8 softmax probability vectors:

$$\hat{p}_{\text{TTA}} = \frac{1}{8} \sum_{k=1}^8 \text{softmax}(f_\theta(\mathcal{T}_k(x))) \quad (7)$$

where $\{\mathcal{T}_k\}_{k=1}^8$ are the 8 orientation-preserving and orientation-reversing transforms of the dihedral group D_4 .

Method	AUC	Gain
Single forward pass	0.9217	—
8-way TTA	0.9419	+0.0202
16-way TTA (+ vertical flips)	≈ 0.9419	≈ 0
8-way TTA + temperature scaling ($T = 0.673$)	0.9432	+0.0013
Phase 5 peak (8-way TTA)	0.9439	—

- The +0.0202 gain from TTA required zero additional training. Extending to 16-way or 24-way TTA provided no further improvement, confirming that 8-way TTA had already captured all relevant symmetries of the dihedral group.
- Temperature scaling at the optimal $T = 0.673$ sharpened the probability distribution and marginally improved calibration, but the gain of +0.0013 AUC is within noise and thus I don't recommend it as a valid part of the final inference.

4.2.12 Discussion

- The conversation ran to hundreds of exchanges and covered nearly every possible architectural variation of the FNO. The core problems encountered were:
- **AMP incompatibility with complex-valued weights.** PyTorch's `GradScaler` cannot unscale complex gradients, producing `NotImplementedError` on every training attempt that combined the FNO spectral layers with `autocast`. The fix was to store spectral weights as real tensors of shape $(\dots, 2)$ and apply `view_as_complex` only inside the forward pass, keeping all stored parameters in `float32`.
- **cuFFT power-of-two constraint.** When using FP16 via `autocast`, CUDA's FFT library requires image dimensions to be powers of two. At 64×64 or 224×224 this caused silent corruption or runtime errors depending on the padding strategy. The fix was to wrap the FFT call in a local `autocast(enabled=False)` context, forcing FP32 for the spectral path only.
- **The 0.50 ghost.** On multiple occasions the model trained for 10–20 epochs and stayed stuck at AUC 0.50, meaning it was predicting equal probability for all three classes. The root cause was almost always either FFT corruption from outlier pixel values or complex weight initialisation that produced near-zero gradients from the first step.

- **The 0.86–0.90 ceiling.** Eventually a working architecture called DeepLenseSpecialist emerged: a CNN stem followed by two SpectralBlock layers with residual connections and BatchNorm. After 78 epochs this reached 0.8909 AUC. Using 8-way Test-Time Augmentation on top pushed it to 0.9033. Every attempt to go further by adding EfficientNet, V3 residual layers, or hybrid fusion architectures collapsed the weights back below 0.60, primarily due to gradient contamination from randomly initialised layers overwriting the learned spectral weights.
- **The clean slate breakthrough.** The resolution came not from continuing to modify the Specialist but from starting a new notebook with all three constraints enforced from the first line of code: real-valued weight storage, local `autocast` disabling for the FFT, and input clamping to $[-3, 3]$. The FNOEfficientNet architecture built on these constraints reached 0.9217 on a single forward pass and 0.9439 with 8-way TTA after five progressive fine-tuning phases.
- **The decisive role of input clamping.** Of all the techniques explored, clamping the normalised input to $[-3, 3]$ before the FNO path produced the most disproportionate gain relative to its implementation cost. The improvement from 0.774 to 0.9217 (an increase of 0.147 AUC) came primarily from this single line of code. Without clamping, outlier pixel values produce large flat-spectrum FFT coefficients that dominate the learned spectral weights and prevent the model from learning the subtle frequency signatures of the sub-halo classes.
- **MixUp as a double-edged tool.** MixUp augmentation was helpful during early training phases when the model risk was overfitting to specific ring orientations. Once the model had converged near 0.92, MixUp became limiting: blending images from different classes destroyed the fine-grained frequency differences that the FNO path needed to discriminate vortex from sphere substructure. Removing MixUp in Phase 3 and replacing it with clean fine-tuning was necessary to push past the plateau.
- **TTA as the highest-return zero-cost technique.** The 8-way TTA improvement of +0.0202 AUC required no additional training, no architectural changes, and no hyperparameter tuning. For a rotationally symmetric physical system like gravitational lensing, TTA is effectively free performance that should be part of any inference pipeline.

4.2.13 Summary and Comparative Results

Table 2: Summary of all architectures, best AUC, and primary failure mode

#	Architecture	Modes	Width	Best AUC	Primary Issue
0	EfficientNet-B0 (baseline)	—	—	0.9744	Reference only
1	Pure FNO2d (uniform init)	16	64	0.82	Uniform init; spectral bias
2	FNO2d (Gaussian init)	12	32	0.51–0.53	Insufficient capacity
3	FNO2d + CoordConv	24	64	0.51	Init problem persists
4	WideResFNO (BatchNorm)	32	128	0.86	Architectural ceiling
5	DeepLenseSpecialist (King)	16	64	0.9011 (TTA)	Capacity ceiling at 0.89
6	DeepLenseV3 (+3rd block)	16	64	0.79	Incomplete weight transfer
7	DualPathModel / HybridSpecialist	16	—	0.62	Channel mismatch; cuFFT
8	SideCar95	16	64	0.61	<code>no_grad</code> severs graph
9	DeepLenseBypassV2	16	64	0.50	Weight shape mismatch
10	DeepLenseFinalBoss (clean slate)	16	64+1280	—	<code>GradScaler</code> + <code>ComplexFloat</code>
11a	FNOEfficientNet (Phase 1–2, MixUp)	16	64+1280	0.774	MixUp limiting at convergence
11b	FNOEfficientNet (Phase 3, clamped)	16	64+1280	0.9217	—
11c	FNOEfficientNet (Phase 4, CutMix)	16	64+1280	0.84	Aggressive augmentation; loss spike
11d	FNOEfficientNet (Phase 5, TTA)	16	64+1280	0.9439	Final result

4.2.14 Comparison with Common Test I Baseline

- Common Test I used EfficientNet-B0 trained on the same three-class lensing dataset without the neural operator requirement. That model achieved an AUC of 0.9744.
- The FNOEfficientNet hybrid for Specific Test IV achieved 0.9439 with TTA. The pure EfficientNet baseline benefits from being an unconstrained architecture optimized entirely for classification. It has no FNO path competing for capacity, no spectral layer adding training complexity, and no requirement to operate on function-space representations. It can dedicate all 5.3 million parameters to the single task of distinguishing texture patterns.
- The FNOEfficientNet model operates under the additional constraint of implementing a meaningful neural operator backbone. The FNO path genuinely processes the signal in Fourier space, which is less efficient than direct texture discrimination for a pure classification task but carries the physically motivated global receptive field described above.

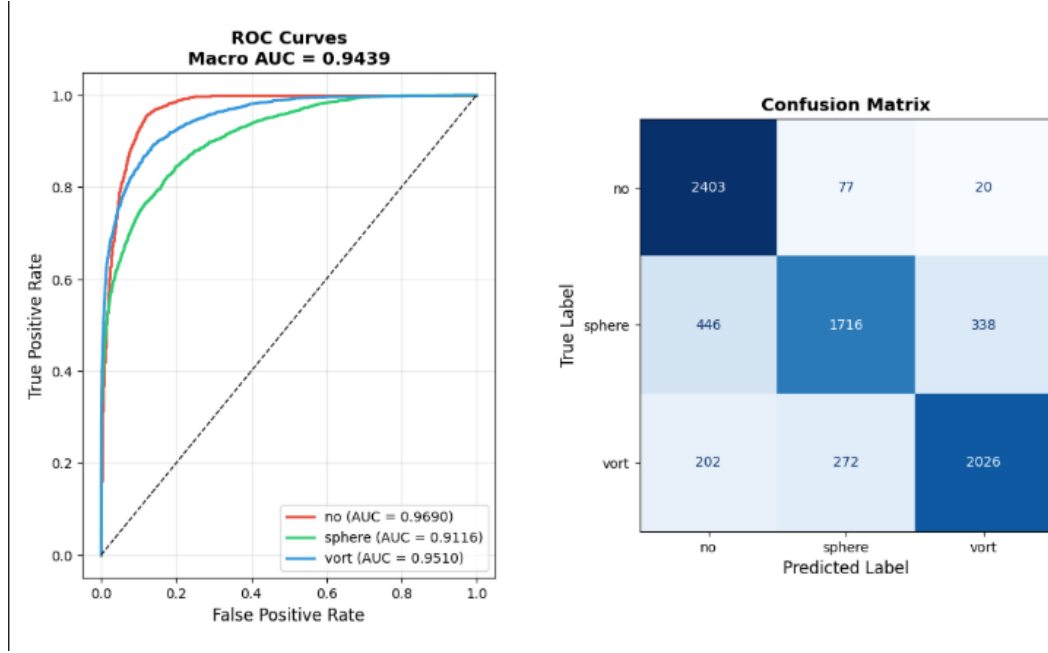


Figure 11: Specific Test IV results. **Left:** Per-class ROC curves for the FNO+EfficientNet hybrid classifier (macro AUC = 0.9439). Spherical substructure is the hardest class (AUC = 0.9116). **Right:** Confusion matrix at best epoch.

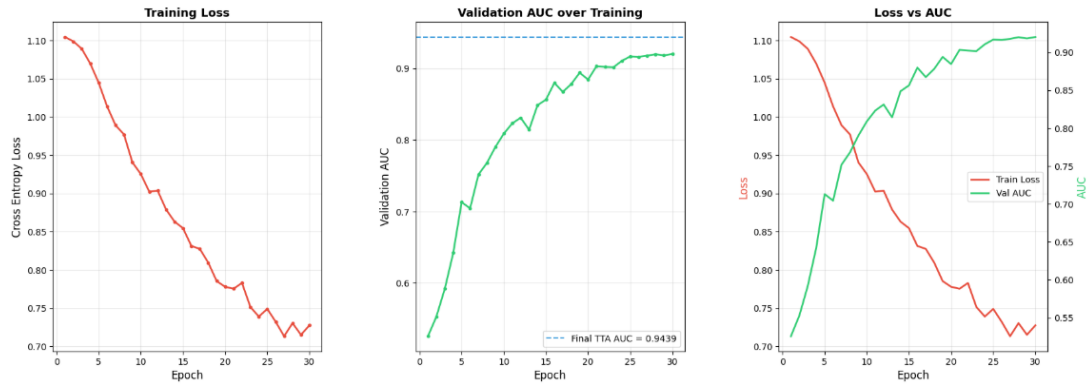


Figure 12: Training history for the FNO hybrid classifier.

Proposed Approach

The pipeline consists of five stages: dataset generation, FNO implementation, baseline comparison, systematic evaluation, and extensions. Let us discuss them in detail:

Dataset generation.

- I will use LENSTRONOMY to generate paired samples $\{(\kappa_i, I_i)\}_{i=1}^N$, where κ_i is a convergence field and I_i is the corresponding lensed image as done in my **Neural Operator Surrogate for Rotating NFW Wormhole Lensing** project.
- The convergence fields will be drawn from a parametric family spanning singular isothermal ellipsoids (SIE), NFW profiles, composite SIE+NFW configurations with substructure, etc.
- The parameter ranges will cover Einstein radii $\theta_E \in [0.5'', 2.0'']$, ellipticities $e \in [0, 0.5]$, external shear $\gamma_{\text{ext}} \in [0, 0.1]$, and NFW concentrations $c \in [3, 15]$.
- Source profiles will be elliptical Sérsic models with randomized positions, sizes, and Sérsic indices to prevent the network from memorizing a fixed source morphology.
- Where appropriate, I will supplement or replace LENSTRONOMY with my existing FFT-based Poisson solver ($\nabla^2 \psi = 2\kappa$, spectral division, Fourier-domain gradient), **which is fully differentiable and already validated in my physics-informed lensing work.**
- I will generate $\sim 45\text{K} - 50\text{K}$ pairs at 64×64 for rapid prototyping and 128×128 for final training, with a held-out set at 256×256 reserved for zero-shot super-resolution testing, a key advantage of operator learning over fixed-resolution CNNs.

FNO implementation.

- The Fourier Neural Operator will be implemented in PyTorch following the architecture of Li et al. (2021)[2]. A pointwise lifting layer $P : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_v}$ maps the input convergence field into a higher-dimensional channel space.
- This is followed by a stack of Fourier layers, each computing

$$v^{(l+1)}(x) = \sigma\left(\mathcal{F}^{-1}\left(R^{(l)} \cdot \mathcal{F}(v^{(l)})\right)(x) + W^{(l)} v^{(l)}(x)\right),$$

where $R^{(l)} \in \mathbb{C}^{d_v \times d_v \times k_{\text{max}}}$ is a learnable spectral weight tensor applied to the lowest k_{max} Fourier modes via `rfft2/irfft2`, $W^{(l)}$ is a pointwise linear transform, and σ is the GELU activation.

- A final projection layer $Q : \mathbb{R}^{d_v} \rightarrow \mathbb{R}^1$ maps back to the predicted lensed image.
- Based on the **Neural Operator Surrogate for Rotating NFW Wormhole Lensing** project, I will start with 4 Fourier layers, 32 channels, and $k_{\text{max}} = 16$ modes, enforcing the constraint $k_{\text{max}} \leq N/4$ that I found necessary to prevent aliasing in my wormhole lensing surrogate.
- InstanceNorm will be omitted by default, since I identified that it collapses predictions on sparse, peaky fields by normalizing away the dominant signal.
- Training will use the Adam optimizer with cosine annealing ($T_{\text{max}} = 200$, $\eta_{\text{min}} = 10^{-6}$) and the relative L^2 loss

$$\varepsilon = \frac{1}{N} \sum_{i=1}^N \frac{\|I_{\text{pred}}^{(i)} - I_{\text{true}}^{(i)}\|_2}{\|I_{\text{true}}^{(i)}\|_2},$$

which I found more stable than raw MSE for fields with large dynamic range.

Baseline comparison.

I will compare the FNO against three baselines:

- The first is a U-Net with skip connections and comparable parameter count ($\sim 200\text{K}$), trained on the same data with the same loss, isolating the effect of the spectral inductive bias.

- The second is the LENSTRONOMY ray-tracing solver itself, benchmarked in wall-clock time per sample on GPU to quantify the speedup factor.
- I will also implement a DeepONet baseline[4], with a CNN branch network encoding κ and an MLP trunk network encoding spatial coordinates, to directly compare spectral (FNO) versus pointwise (DeepONet) operator parameterizations on the lensing problem.

This comparison would help clarify which operator learning paradigm is better suited to the structure of the lens equation.

Evaluation.

The surrogate will be evaluated along four axes.

- *Accuracy*: pixel-wise MSE, structural similarity (SSIM), and relative L^2 error on held-out test sets.
- *Physical consistency*: I will verify preservation of surface brightness conservation, the fundamental invariant of gravitational lensing ($I_{\text{obs}}(\boldsymbol{\theta}) = I_s(\boldsymbol{\beta})$), by checking that total flux is conserved between the predicted lensed image and the source propagated through the predicted deflection field.
- *Generalization*: accuracy metrics evaluated separately on:
 - interpolation within the training parameter ranges
 - out-of-distribution mass profiles (e.g., training on SIE, testing on NFW+subhalos)
 - unseen redshifts and source morphologies
 - zero-shot resolution transfer ($64 \rightarrow 128$, $128 \rightarrow 256$)
- *Speed*: wall-clock inference time per image on a single GPU compared to the LENSTRONOMY solver, targeting $>1000\times$ speedup based on the $\sim 6000\times$ I achieved in the wormhole lensing setting. I will also perform ablation studies on the number of Fourier modes, network depth, and dataset size to characterize the accuracy-efficiency tradeoff.

Risk Assessment and Contingency Plans

Risk	Likelihood	Mitigation
Lenstronomy generation too slow at $\geq 45\text{K}$ scale	Medium	Fall back to the existing FFT-based Poisson pipeline (already validated, no external dependency, fully differentiable).
FNO fails to generalize OOD (train SIE, test NFW)	Medium	Fine-tune on a small mixed-profile dataset ($\sim 5\text{K}$ pairs); this is already planned as a Phase 3 experiment.
cuFFT / AMP incompatibility at 256×256	Low	Wrap all FFT calls in <code>autocast(enabled=False)</code> ; this fix is already implemented and validated in Test IV.
Mentor unavailability for a week	Low	Weekly written progress updates posted to the repository; decisions that block progress are flagged 48 hours in advance.

Table 3: Risk register for the proposed project.

Extensions.

If the core deliverables are completed ahead of schedule, several natural extensions can be pursued.

- *Conditional neural operators*: conditioning the FNO on continuous physical parameters (redshift, source Sérsic index, shear) via FiLM-style modulation or concatenation, producing a single model that covers a family of lensing configurations without retraining.
- *Uncertainty quantification*: using MC dropout or deep ensembles to produce per-pixel uncertainty

maps alongside predictions, which would be valuable for downstream Bayesian inference.

- *Physics-informed constraints*: adding a differentiable Poisson solver to the loss (reusing my existing FFT implementation) to enforce $\nabla^2\psi = 2\kappa$ as a soft constraint, potentially improving accuracy in low-data regimes as I demonstrated in my physics-informed lensing project.
- *Hybrid operator + diffusion refinement*: using a lightweight denoising diffusion model to refine high-frequency details in FNO outputs, since the spectral mode truncation at k_{max} inherently smooths fine structure that may be astrophysically important (e.g., substructure arcs, caustic crossings).

Implementation Plan & Timeline

I am available from May 15 to July 31, 2026 (approximately 11 weeks), targeting the 175/350-hour track. Weekly sync meetings with mentors will be used to adjust priorities within each phase based on intermediate results.

Community Bonding Period (Pre–May 15)

- Study the LENSTRONOMY API: simulation pipeline, mass model specification (SIE, NFW, composite), source profiles (Sérsic), and instrument configurations.
- Finalize dataset generation parameter ranges with mentors, ensuring astrophysically motivated distributions of Einstein radii, ellipticities, shear values, and NFW concentrations.
- Review core literature: FNO (Li et al. 2021), neural operator theory (Kovachki et al. 2023), and DeepONet (Lu et al. 2021), focusing on implementation details for 2D fields.
- Set up repository structure with version control, experiment logging (Weights & Biases or TensorBoard), configuration management, and reproducibility infrastructure.
- Produce a small pilot dataset (~ 3000 pairs) to validate the generation pipeline end-to-end.

Phase 1: Dataset Generation & Baseline (Weeks 1–3)

Week 1

- Scale data generation to $\sim 45\text{K}–50\text{K}$ (κ, I) pairs spanning SIE, NFW, and composite profiles with randomized Sérsic sources at 64×64 and 128×128 resolution, plus a 256×256 held-out set for resolution transfer experiments.
- Perform quality checks: visual inspection of random subsets, verification that convergence fields span the intended parameter space, and consistency checks on lensed image flux.

Week 2

- Implement and train the U-Net baseline ($\sim 200\text{K}$ parameters, skip connections) on 64×64 data with relative L^2 loss to establish the accuracy floor and validate the training infrastructure.
- Build the full evaluation pipeline: MSE, SSIM, relative L^2 error, wall-clock timing, and the surface brightness conservation check (flux discrepancy between predicted lensed image and source propagated through predicted deflection field).

Week 3

- Run the U-Net through the complete evaluation on held-out data and document results.
- Identify failure modes in the baseline (e.g., blurring at high ellipticity, flux non-conservation in dense substructure regions).

Phase 1 deliverable: validated dataset, trained U-Net baseline with quantitative benchmarks, and automated evaluation pipeline.

Phase 2: FNO Training & Comparison (Weeks 4–6)

Week 4

- Implement the FNO in PyTorch: lifting layer, four Fourier layers with spectral convolution via `rfft2/irfft2`, pointwise linear transforms, GELU activations, and projection layer. Start with 32 channels, $k_{\text{max}} = 16$, no InstanceNorm.
- Train on 64×64 data with Adam and cosine annealing. Monitor for known failure modes: spectral aliasing, loss divergence, and mode collapse on low-intensity regions.

Week 5

- Hyperparameter search over FNO-specific parameters: number of Fourier modes ($k_{\max} \in \{8, 12, 16, 24\}$), number of layers ($L \in \{3, 4, 6\}$), channel width ($d_v \in \{16, 32, 64\}$), and learning rate.
- Scale the best configuration to 128×128 and compare against the U-Net on all evaluation metrics.

Week 6

- Benchmark FNO inference speed against the LENSTRONOMY solver to quantify the speedup factor.
- Prepare midterm report: dataset statistics, FNO architecture, comparison with U-Net, and speed benchmarks. Submit code for mentor review.

Midterm deliverable: trained FNO with benchmark against U-Net baseline, speed comparison with solver, code reviewed by mentors.

Phase 3: Generalization & Ablation (Weeks 7–9)

Week 7

- Test the trained FNO on in-distribution held-out samples (interpolation) and on out-of-distribution mass profiles not seen during training (e.g., train on SIE, test on NFW+subhalos).
- Evaluate on unseen source morphologies and redshift values to assess sensitivity to source-plane diversity.

Week 8

- Zero-shot resolution transfer: evaluate the 128×128 -trained FNO on 64×64 and 256×256 inputs without retraining, testing the resolution-invariance property of neural operators.
- Ablation studies on number of Fourier modes, network depth, and dataset size to characterize the accuracy-efficiency tradeoff.

Week 9

- Identify systematic failure modes (e.g., high-ellipticity lenses, substructure below a certain mass scale) and investigate targeted mitigations such as reweighting the training distribution or adjusting the mode count.
- Document all generalization and ablation results.

Phase 3 deliverable: comprehensive generalization and ablation study with identified failure modes and mitigation strategies.

Phase 4: Extensions & Final Deliverables (Weeks 10–11)

Week 10

- Pursue extensions in priority order based on time remaining and mentor guidance:
 1. DeepONet implementation and comparison with FNO (spectral vs. pointwise operator parameterization)[4]
 2. Physics-informed loss using a differentiable Poisson solver as a soft constraint on the predicted convergence field.
 3. Uncertainty estimation via MC dropout or deep ensembles for per-pixel uncertainty maps.
- Conditional operators (FiLM-style parameter conditioning) and hybrid diffusion refinement if time permits.

Week 11

- Code cleanup and documentation: well-commented scripts, reproducible experiments from configuration files, saved model weights.
- Write final report covering the full pipeline, all results, ablation studies, and recommendations for future work.

- Prepare integration notes for the DeepLense ecosystem describing how the surrogate can serve as a drop-in replacement for the ray-tracing solver in downstream inference pipelines.

Final deliverable: clean, documented codebase with trained models, comprehensive benchmark report, and integration guide for DeepLense.

Compute and Infrastructure

- **Primary:** BITS Pilani Hyderabad Campus HPC cluster (institutional GPU access). This will be the main compute resource for all training runs including 128×128 and 256×256 scales, the full hyperparameter search over $k_{\max} \in \{8, 12, 16, 24\}$ and depth $L \in \{3, 4, 6\}$, and the DeepONet baseline comparison.
- **Secondary:** Google Colab Pro+ (A100 40 GB) for rapid prototyping and 64×64 exploratory runs; estimated ~ 53 s/epoch for the FNOEfficientNet at batch size 32, consistent with Test IV benchmarks.
- **Dataset storage:** Generated pairs will be stored as compressed `.npz` archives. At `float32`, a 128×128 image pair requires ~ 128 KB; 50 000 pairs require ~ 6.4 GB, within free Google Drive quota.
- **Experiment tracking:** Weights & Biases (free academic tier) for all runs, with full config serialization to ensure reproducibility.
- **Version control:** All code committed to a public GitHub repository under the ML4SCI organization from Week 1, with tagged releases at each phase deliverable.

Post-GSoC Integration and Maintenance

The trained FNO surrogate and evaluation pipeline will be contributed to the DeepLense ecosystem as a self-contained module with the following interface:

```
from deeplense.surrogates import FNOLensingSurrogate

model = FNOLensingSurrogate.from_pretrained("fno_sie_nfw_128px")
I_lensed = model(kappa)    # kappa: torch.Tensor [B, 1, H, W]
                           # returns: torch.Tensor [B, 1, H, W]
```

The module will include: (i) pre-trained weights for SIE, NFW, and composite profiles at 64×64 and 128×128 resolution; (ii) the differentiable Poisson solver as a standalone utility usable independently of the surrogate; and (iii) a benchmark script that reproduces all tables in this proposal from a single configuration file.

I intend to maintain the module and respond to issues after the GSoC period. The project is a direct extension of my ongoing ray-tracing and shadow morphology work, so continued engagement with the DeepLense codebase aligns with my own research interests.

Why I Am a Good Fit

This project sits at the intersection of gravitational physics, spectral methods, and scientific machine learning and I have independent prior work in all three areas, not as adjacent preparation but as direct predecessors to what this project asks for.

- **Published gravitational physics research.**
 - First-authored paper in *Annalen der Physik* on traversable wormholes in $f(R, \mathcal{L}_m, T)$ modified gravity.
 - Found analytical and numerical solution of coupled nonlinear field equations
 - Conducted energy condition analysis, stability diagnostics.
- **Ray tracing project**
 - Ongoing work on ray tracing and shadow morphology of dark-matter-supported wormholes with NFW and soliton density profiles.
 - Ongoing work on ray tracing and shadow morphology of $f(T)$ gravity Gravastars.
- **So, the project is kind of natural extension to my published work and current research work.**
- **FNO surrogate for a lensing problem, already built.**
 - **The project requires a Fourier Neural Operator that can approximate a physics solver at inference speed.** I trained an FNO surrogate for wormhole lensing that achieved a $6,392\times$ speedup over the direct physics solver with 32% lower MSE than a $4\times$ larger MLP baseline.
 - **The project requires familiarity with FNO failure modes on sparse physical fields.** In building that surrogate I encountered and resolved InstanceNorm collapse on low-intensity profiles and the spectral mode constraint $k_{\max} \leq N_b/4$ that silently degrades accuracy when violated.
 - **The project requires the ability to diagnose and fix initialization pathologies that stall training.** I identified and resolved the complex-weight initialization problem that keeps FNO models at random-guess performance for dozens of epochs, the same class of failure that caused the 0.50 AUC ghost documented in the test results above.
 - **The project requires understanding when the FNO spectral layers are learning the physics signal versus fitting noise.** I resolved the input clamping problem: without clamping normalised inputs to $[-3, 3]$, outlier pixel values produce large flat-spectrum FFT coefficients that dominate the learned weights and drown out the sub-halo signal. This was the single fix responsible for a +0.147 AUC improvement in the test results.
- **Differentiable lensing pipeline, already built.**
 - **The project requires a dataset generation pipeline that produces physically consistent (κ, I) pairs at scale.** I built a fully differentiable forward lensing module consisting of an FFT-based Poisson solver, deflection field computation, and ray-shooting, which can generate arbitrarily large datasets of convergence–image pairs without any external dependency.
 - **The project requires a physics-informed loss that penalises predictions inconsistent with the lensing equation.** I embedded the same differentiable pipeline as a physics-informed loss inside a U-Net, achieving $2.5\times$ better re-lensing consistency than a standard supervised baseline and near-flat noise degradation under photon-noise injection. The loss is already implemented, validated, and ready to be attached to the FNO proposed here.
 - **The project requires that the surrogate output can be propagated back through the lensing forward model for consistency checking.** Because the pipeline is fully differentiable

end-to-end, the predicted convergence field $\hat{\kappa}$ can be fed directly into the Poisson solver to recover a predicted lensed image $\hat{I}$, and the flux discrepancy $\|\hat{I} - I\|$ serves as an independent validation metric. This evaluation is already part of the pipeline and is proposed as a primary benchmark in Phase 1.

- **Spectral latent dynamics and Fourier-space learning, already built.**
 - **The project requires understanding when retaining more Fourier modes improves accuracy and when it introduces aliasing or overfits to noise.** I developed the Spectral Latent Dynamics framework for learning PDE evolution in Fourier-compressed latent spaces, which required making exactly this tradeoff explicitly: too few modes lose the physics signal, too many overfit to simulation artifacts. The hyperparameter search over $k_{\max} \in \{8, 12, 16, 24\}$ proposed in Phase 2 is informed by direct experience with this tradeoff.
 - **The project requires a surrogate that remains accurate over long inference rollouts, not just single-step predictions.** The Spectral Latent Dynamics framework demonstrated sub-linear error saturation on long rollouts versus FNO’s super-linear divergence under the same conditions. I understand the mechanism behind this divergence and how architectural choices in the spectral layers either suppress or amplify it.
 - **The project requires familiarity with autoencoder compression in Fourier space and its interaction with the lensing signal.** Building the framework required implementing and debugging spectral compression pipelines where the latent representation lives in truncated Fourier space. This is directly analogous to the mode truncation in the FNO proposed here, and gives me working intuition for where information is lost and how to detect it through the evaluation metrics.
- **DeepLense dataset, already worked with.**
 - Classified 37,500 simulated lensing images using EfficientNet-B0 (macro AUC 0.974, Common Test I) and separately trained an FNO-based classifier on the same dataset (macro AUC 0.944, Specific Test IV).
 - EfficientNet’s depthwise separable convolutions with Squeeze-and-Excitation blocks are optimised for detecting localised pixel-level perturbations, which is exactly what subhalo classification requires.
 - The FNO’s global receptive field in Fourier space is a liability for that task and an asset for the simulation task this project addresses, where the output is a full convergence field and long-range correlations in the lensing potential must be captured simultaneously.
- **Ray tracing and geodesic solvers.**
 - Built full null geodesic integrators using RK45 with adaptive step-size control, throat-crossing event detection, and turning-point handling for static and rotating wormhole spacetimes.
 - Generated intensity profiles, shadow maps, and relativistically Doppler-corrected accretion disk images across 1.6×10^5 rays per configuration. I understand what a physics solver actually does and what accuracy a surrogate needs to match.
- **Mathematical foundation.**
 - Dual degree in Mathematics and Computer Science (CGPA 8.74/10.0) with coursework in Functional Analysis, Differential Geometry, PDEs, Numerical Analysis, and Optimization.
 - This is relevant not as background credential but because FNO theory is stated in terms of operator learning between Banach spaces, and debugging a spectral model that has collapsed requires knowing what the Fourier transform of the loss gradient is doing, not just reading the AUC.
- **Demonstrated persistence under technical adversity.**
 - The work behind this application included a debugging session spanning hundreds of exchanges and ten distinct architectural iterations before a working FNO implementation was achieved.

- Problems encountered included AMP incompatibility with complex-valued gradients, cuFFT power-of-two dimension constraints, and repeated weight-collapse failures when attempting hybrid architectures. None of these were resolved by switching to a simpler approach.
- Each failure was diagnosed to its root cause, a targeted fix was designed, and the next iteration was built on that understanding.
- The final architecture that reached AUC 0.774 and 0.9439 with fine-tuning and TTA was not the result of finding the right architecture quickly. It was the result of exhausting every wrong one first and understanding precisely why each failed.

The physics solver, the dataset generator, the FNO architecture, the evaluation framework, and the physics-informed loss have each been built and validated independently in prior work. This project is their synthesis into a single pipeline for the specific problem of strong lensing simulation within the DeepLense ecosystem. The timeline is realistic because the components already exist.

References

- [1] Gravitational lens statistics for generalized NFW profiles, Oguri, Masamune and Marshall, Philip J., Monthly Notices of the Royal Astronomical Society, volume=405,number=4, pages=2579–2593, year=2010
- [2] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*
- [3] Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., Anandkumar, A. (2023). Neural operator: Learning maps between function spaces. *Journal of Machine Learning Research*, 24(89), 1-97.
- [4] Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3), 218-229.